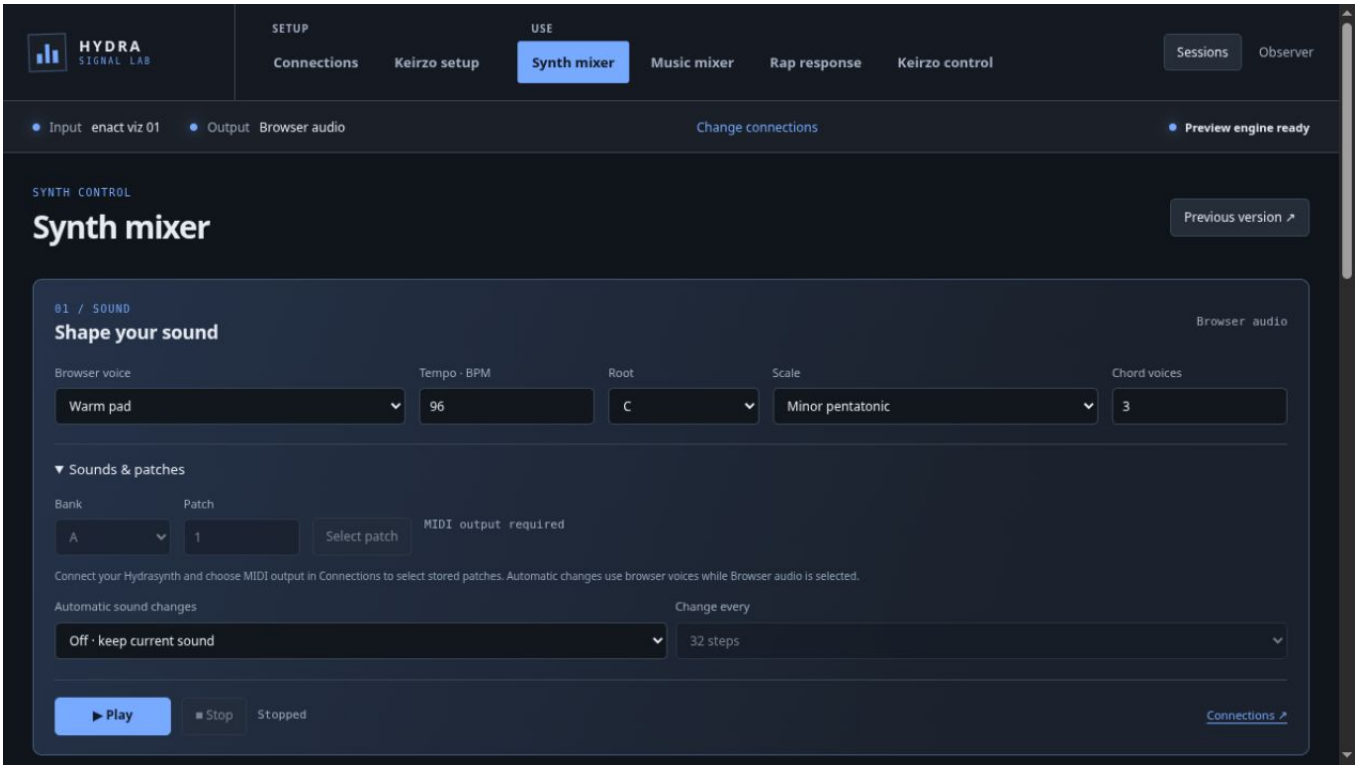
OPERATOR GUIDE

Hydra Signal Lab instruction manual

Linux setup and operation with Keirzo and Hydrasynth

Linux edition | 20 October 2026

Start with browser sound and bundled recordings, then add generated music, MIDI instruments and robot performance as needed. This manual covers first installation, daily use, six-laptop provisioning, study collection and recovery.



Screenshots show the actual local application with the robot and Hydrasynth disconnected. Empty results and disabled hardware controls are intentional. Physical procedures are based on the current controls and implementation; they have not been demonstrated on a connected Linux rig for this edition. No paid generation was performed to create the screenshots.

Quick start first. Detailed procedures follow. Use the linked contents or PDF bookmarks to jump to a chapter.

Contents

Quick start your first sound without hardware 3

Installation and launch reference 4

Provisioning six laptops and API credentials 7

Connections choose input and sound output 9

Signal mixes turn data into reusable musical controls 12

Synth mixer play and shape browser or Hydrasynth sound 15

Music mixer create instrumental tracks and shape their stems 18

Rap response record type perform and reuse replies 22

Saved sessions presets and moving a setup 25

Observer inspect telemetry and diagnose behavior 28

Keirzo hardware connections and first checks 31

Stopping torque and disconnecting 33

Manual rib drum and LED tests 35

Head calibration and preparation 37

Keirzo control coordinated lights ribs and head 39

Automatic drumming pattern timing and recovery 41

Native Hydrasynth MIDI Linux audio and older routes 43

Daily operation updates and saved material 46

Study recording and participant handovers 48

Collecting exporting and combining study data 50

Troubleshooting and verification 52

Reference glossary and verification 55

CHAPTER 01

Quick start your first sound without hardware

You can use the dashboard before connecting either the robot or Hydrasynth. The browser can make sound from simulated signals or the three bundled movement recordings. The six saved voice demonstrations also play without an API key. New music and rap generation need a configured ElevenLabs account and internet access; the first sound exercise below uses neither.

The five-minute walkthrough begins after installation. A first installation can take longer because it downloads Python packages and a local speech model. The Ubuntu/Debian installer does not connect motors, send physical MIDI commands or record an audio input during setup.

Install once on Ubuntu or Debian

Open a terminal. Install Git if it is missing, then clone the published repository. If it is private, use the GitHub authentication provided by the repository owner. The owner must commit and push the installer and application changes before a new laptop can receive them through Git.

```
sudo apt-get install git
git clone https://github.com/richardsavery/hydrasynth.git
cd hydrasynth
bash tools/setup.sh
```

Run the setup command as your ordinary desktop user. It asks for your sudo password only if system packages are missing. At the hidden API key prompt, paste this laptop’s key or press Enter to skip. No characters are displayed while pasting. Once setup finishes, start the application:

```
make dashboard-sonify
```

Hear something and try a signal

1. Leave the terminal open. In the same laptop’s browser, open http://127.0.0.1:8790/#synth-mixer.
2. Open Setup > Connections. Set Synth output to Browser audio and press Test sound. Check the computer’s output device and volume if you hear nothing.
3. Open Use > Synth mixer. In Signal mixes, set Signals from to Simulated demo signals. This supplies a changing input without a recording, live stream or robot.
4. Choose Movement as Input 1 in Mix A. In the Filter cutoff card, set Signal mix to Mix A. Press Play in Shape your sound: the cutoff should now follow the simulated movement. Try the sound and tempo controls, then press Stop. A parameter left on Manual will not follow a signal.
5. To try recorded movement, select a bundled recording in Connections and return to the mixer with Signals from set to Connections input. Keep the input source and your mappings consistent.
6. For a no-cost voice preview, open Use > Rap response, expand Voice demos and play a saved demonstration. Listening does not choose a new voice or generate a rap.

Tabs are workspaces within one application. Navigating between them can leave playback and robot control running; use the appropriate Stop control when you want to end an activity. Loading or viewing a page does not start robot movement.

Stop and return later

Stop playback in the dashboard. If you are collecting study data, wait for the researcher Study panel to report Saved locally before ending the server. In the terminal running the dashboard, press Ctrl+C and let shutdown finish. To return later, open a terminal in the hydrasynth folder, run make dashboard-sonify and open the same address. Installation does not create a login service or background launcher.

The application records study events automatically unless explicitly disabled for a development run. Read the study chapter before involving participants. The ordinary participant interface hides the researcher controls; it does not disable recording.

CHAPTER 02

Installation and launch reference

What you need

The automated installer targets Ubuntu/Debian Linux. It requires Python 3.10 or newer, a normal user account and internet access for missing packages and the initial model download. Allow several GB of free space for the environment, download caches, recordings and generated audio. Speakers or headphones are enough for browser playback; a microphone is needed only when you choose to record. Node.js is optional for running the application and is needed for the JavaScript test suite.

Component	Purpose and installation
build-essential, pkg-config, libasound2-dev	Build the native C tools against Linux ALSA.
alsa-utils	Device diagnostics, including aconnect and arecord.
python3-venv and .venv	Create an isolated Python environment for this checkout.
requirements-robot.txt	Install serial and Dynamixel support for the robot.
requirements-rap-response.txt	Install local transcription and vocal processing dependencies.
FFmpeg and ffprobe	Prepare, inspect and validate audio files.
Whisper Small in var/whisper/small	Transcribe microphone recordings locally after initial download.
Node.js	Run JavaScript regression tests; not required to serve the dashboard.

Setup reuses an existing usable .venv, installs the two requirements files, builds hydractl and hydra-audio, runs their self-tests, prepares the model and checks that it loads on this CPU. Existing credentials and local data are retained. Setup does not make a paid provider request, so successful installation alone does not prove that the ElevenLabs account can access every generation feature.

Installer options

```
bash tools/setup.sh --skip-model --no-key
bash tools/setup.sh --skip-system
PYTHON=/path/to/python3 bash tools/setup.sh
bash tools/setup.sh --help
```

Use --skip-model for a quicker preview installation that does not yet need microphone transcription. Typed rap input remains available, but generation still requires a provider key. Use --no-key to leave credential provisioning until later. Use --skip-system only when an administrator has already installed the required packages, or you have provided equivalent build/audio dependencies. It skips apt and sudo, not the dependency checks, Python installation or build.

The installer uses /usr/bin/python3 by default when creating a new environment, avoiding an unconfigured asdf shim. PYTHON selects an alternative for a new environment; an existing .venv retains its original interpreter. A broken or too-old environment is left intact with an explanatory error. Move it aside and recreate it with a suitable interpreter rather than copying an environment from another computer. The repository’s .tool-versions selects Python 3.12.9 and Node.js 22.20.0 for asdf users.

Equivalent manual installation

```
sudo apt-get install build-essential pkg-config libasound2-dev \
  alsa-utils python3-venv ffmpeg ca-certificates
/usr/bin/python3 -m venv .venv
.venv/bin/python -m pip install -r requirements-robot.txt -r requirements-rap-response.txt
make all
.venv/bin/python tools/prepare_rap_transcription.py
.venv/bin/python tools/configure_api_key.py
```

On another Linux distribution, install its equivalent packages and use --skip-system. A matching ALSA development package can also be extracted into var/native-deps on a machine without installation privileges; docs/linux.md describes this fallback. Normal package installation is the simpler arrangement for a fleet of laptops.

Choose the launch mode

Command	Use it for
make dashboard-sonify	Everyday browser work with no native Hydrasynth required. It starts the server in demo mode.
make dashboard	The connected native Hydrasynth service, incoming telemetry and local command broker. Requires the intended MIDI device.
make dashboard-sonify DASHBOARD_ARGS='--port 8791'	A different local port. Open that port in the browser; browser storage is separate for each origin.
make dashboard-sonify DASHBOARD_ARGS='--study-dir /path/to/local/study-data'	An explicit study storage location. Use the same path consistently.
make dashboard-sonify RECORDINGS_DIR=/path/to/recordings	Use another recordings library instead of the bundled default.

The direct equivalent of the normal no-hardware launch is .venv/bin/python tools/hydra_dashboard.py --demo. Extra server options can be appended there or supplied through DASHBOARD_ARGS. Keep the default loopback host, 127.0.0.1, for normal operation. Each laptop runs its own server; opening its address on another laptop will address that other laptop instead. The local audio and study APIs enforce local access, so changing the bind address is not a turnkey shared server deployment.

macOS and other platforms

On a Mac, finish Apple’s Command Line Tools installation if needed, then run bash tools/setup.sh in the clone. The Mac installer prepares Homebrew, FFmpeg, Python dependencies, native CoreMIDI/Core Audio tools and Whisper Small. It preserves existing keys, model files and local data. Use the same setup command after git pull for updates, then make dashboard-sonify to start. See docs/macos.md for the complete Mac procedure and permissions. Native Windows requires a separate port and is not supported.

CHAPTER 03

Provisioning six laptops and API credentials

Prepare a reproducible set of machines

1. Publish the application version you intend to use: commit and push the required repository changes before cloning the other laptops. Record the chosen Git revision for the session.
2. Label the physical laptops, for example laptop-01 through laptop-06. Clone the same repository revision on each and run setup as the user who will operate the application.
3. Provision that laptop’s credentials separately from Git. Let its first dashboard run create its own study database and installation ID.
4. Use the same browser and address convention on each laptop. Test browser sound, the bundled recordings and voice demos; then grant microphone or Web MIDI permission only for the features you need.
5. Record each physical label alongside the installation ID displayed in the researcher Study panel. Synchronize laptop clocks before data collection.
6. If hardware will be used, complete the physical connection and calibration checks on the actual rig before participant use.

Recommended credential arrangement

For laptops you manage under trusted organiser accounts, provision one restricted ElevenLabs key per laptop, named hydrasynth-laptop-01 through hydrasynth-laptop-06. Separate keys let you replace or revoke one machine’s credential independently. The provider supports endpoint restrictions and per-key credit limits; usage still counts against the workspace quota. Service account keys are intended for backend workloads when available in your workspace. Check the current account controls at https://elevenlabs.io/docs/overview/administration/workspaces/api-keys.

Distribute credentials privately through a password manager or a USB drive you control, then enter the appropriate key in the hidden installer prompt. The application’s Python backend uses the key; it is not intended to be pasted into dashboard fields, browser JavaScript, source code or a shared chat. A person who controls the laptop account can nevertheless read its local files. Owner-only permissions protect against other ordinary accounts, not against the machine’s owner or administrator.

For somebody else’s own computer, use their own authorised provider credential, or arrange a central authenticated generation service if they must not possess your provider secret. This repository currently makes provider calls from each local Python backend and does not implement that central service. A single shared key can work technically, but it removes independent revocation and gives every recipient the same credential.

Set replace or transfer a key

```
.venv/bin/python tools/configure_api_key.py
.venv/bin/python tools/configure_api_key.py --replace
```

The first command keeps an existing saved key; --replace prompts for a replacement. Paste the key alone, without quotation marks, spaces or line breaks. Press Enter to skip without changing the existing file. In a noninteractive terminal, configuration is skipped instead of echoing a secret. The helper writes the project's .env atomically with mode 600, meaning owner read/write only, and preserves other settings. .env is Git-ignored; .env.example is a blank template, not a place to put a real key.

To provision a prepared private file, use the following pattern from the checkout, substituting the actual USB path. This replaces .env, so use the prompt instead when other existing settings must be preserved.

```
install -m 600 /media/USB/laptop-01.env .env
```

A private provisioning file contains the setting ELEVENLABS_API_KEY followed by an equals sign and that machine's key. Restart a running dashboard after changing it. An ELEVENLABS_API_KEY already set in the process environment takes precedence over .env; update or unset it if a newly saved key appears to have no effect. Never put a real key in Git, even in a private repository. Replace a disclosed key through the provider and update the affected laptop.

What consumes credits

New music generation and new raps use the provider. The rap pipeline can include lyric planning, music generation, stem separation and voice conversion. Grant the endpoint access needed by the features you plan to use and confirm account access before a session. Local transcription, local pitch/autotune edits, browser synthesis, saved voice demos and playback of already saved audio do not need new provider requests. Canceling an in-flight generation cannot guarantee that no credits were consumed. In Music mixer, set Next track to Loop before playing a saved track if you want to avoid automatic replacement generation.

Save time on the speech model

Download Whisper Small on one machine, then copy only the complete var/whisper/small directory to the same relative location in the other clones before running setup. The model used here is approximately 464 MiB. Setup recognises complete model files and checks that they load locally. A copied model avoids repeated downloads but does not replace installation of the Python transcription dependencies.

```
# On each new clone, using the actual USB path:
mkdir -p var/whisper
cp -a /media/USB/small var/whisper/
bash tools/setup.sh
```

Create .venv and build native binaries separately on every laptop. Do not clone another machine's complete var directory or study database: that can duplicate its installation identity, recordings and private data. Transfer head-calibration.json only for the same physical robot and review the calibration before powered operation. Copying the model, provisioning the key and transferring robot calibration are three separate tasks.

CHAPTER 04

Connections choose input and sound output

Connections is the starting point for choosing movement data and the destination for Synth mixer sound. Open Setup > Connections, or http://127.0.0.1:8790/#connections. Input data and sound output are separate choices: a recording can drive a real Hydrasynth, and live movement can drive browser audio. You do not need an API key, robot or synth to use the bundled recordings, browser synth or voice demos.

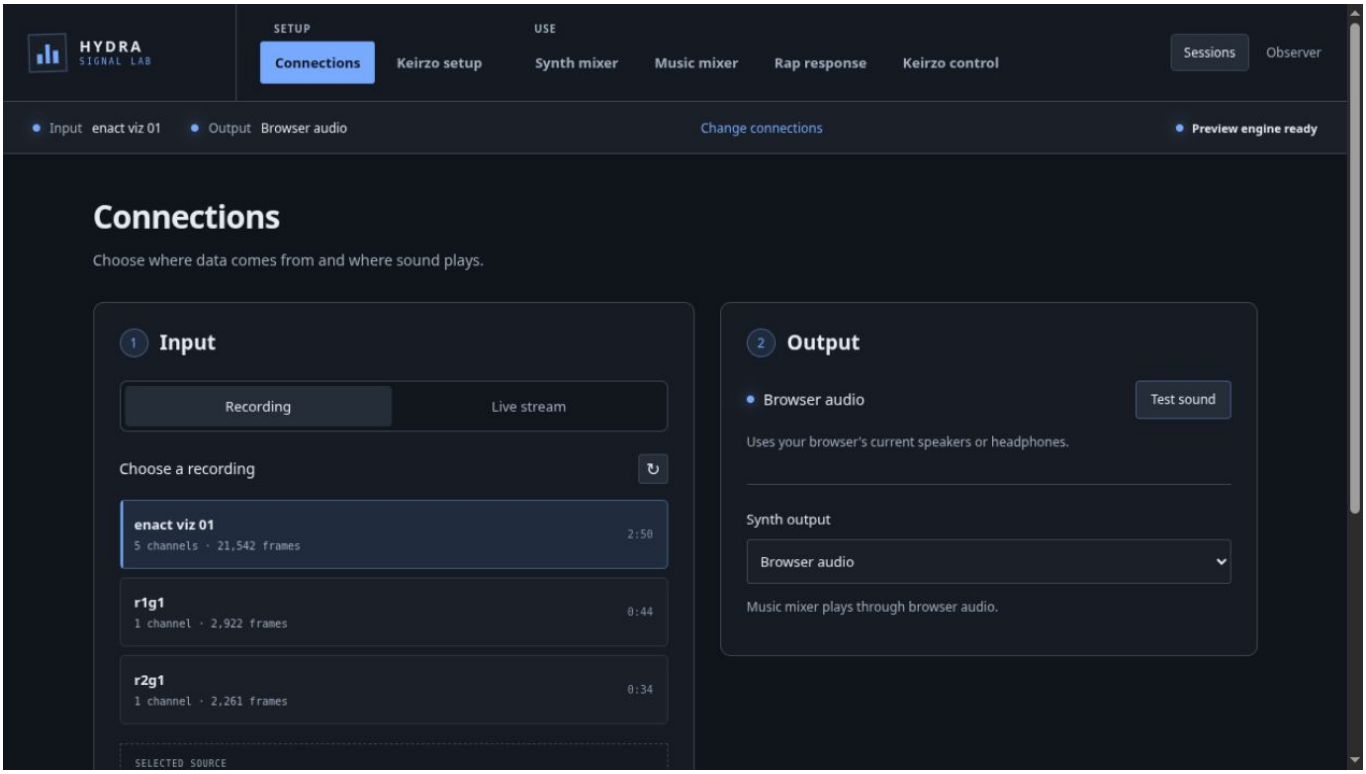


Figure 1. Connections separates the data source on the left from sound output on the right. The browser-only path works with the robot and synth disconnected.

Start with a bundled recording

1. In Input, choose Recording. Wait for the list to load. A normal clone includes three compressed recordings; the dashboard reads these directly, so do not unzip them.
2. Select a recording. Check its title and the input summary near the top of the dashboard. If you added files while the server was running, use Refresh recordings beside Choose a recording.
3. In Output, leave Synth output set to Browser audio. Press Test sound and listen through the computer’s selected speakers or headphones. The status should report that the test sound played.
4. Open Use > Synth mixer. Choose Connections input under Signals from when you want the selected recording to drive mappings. Select a field in a Signal mix, map it to a parameter, then press Play.

5 Return to Connections whenever you need another recording or output. Merely opening this workspace does not stop a mixer that is playing. Use the mixer’s Stop control when you want to stop.

Recordings contain numerical data, not prerecorded music. Selecting one does not play a soundtrack. Music mixer uses the same data as a control source but needs an existing or newly generated track for its sound.

Check browser sound before investigating MIDI

Test sound uses the browser’s current audio destination. Select the desired device in Linux sound settings and check application and device volumes. Its status confirms scheduled playback, not audible speaker output. Check whether the browser tab is muted if the test is silent.

Music mixer and Rap response play through browser audio. Changing Synth output to MIDI device changes the Synth route; it does not send generated music to the synth. The Hydrasynth needs its own audible output path, such as headphones or an audio connection to an interface. MIDI carries performance commands, not the synth’s recorded audio.

Use a live movement stream

1 Start the movement-data sender on the robot or replay computer. Obtain its WebSocket address. For a sender running on this same laptop, the standard address is ws://localhost:8765.

2 In Connections, select Live stream. Enter Stream address and press Connect live. Allow browser local-network access if prompted for a network sender.

3 Wait for usable fields to appear in the live monitor. A connected socket or a greeting alone is not movement data. Check the message rate, freshness and a changing numeric signal before opening a mixer.

4 Choose a channel and signal in the monitor to inspect its graph. Position and velocity are separate inputs. For arrays or tensors, the monitor shows a reduced value and a bounded preview.

5 In either mixer, set Signals from to Connections input and choose the discovered fields. Start playback explicitly. Keep Reconnect automatically selected if you want the browser to reconnect after a socket interruption.

6 Use Disconnect to end the live connection. Use Return to recording, then choose the required recording, when returning to recorded data.

Pause display freezes the inspection display only. The browser still receives movement, learns ranges and drives active mappings. Reset learned ranges changes calibration, not the sender. Use it after a change in movement range, not as a substitute for reconnecting a broken stream. A channel becomes stale when it has not supplied usable data for more than three seconds; updates on other channels do not make its old values current.

A live socket closure stops Synth playback. Automatic reconnect does not restart a performance; press Play again after the stream returns. An open connection with a temporarily stale required channel waits for fresh values. A connected light does not mean every mapped input is valid.

Symptom	What to check
Cannot connect	Confirm the sender is running, its host and port match, and the laptop can reach it. A generic WebSocket error does not distinguish a network, permission, TLS or sender problem.
Connected, no usable data	Start replay or the robot data feed. Status greetings contain no values. Open Troubleshooting for invalid-message reasons and a valid JSON example.
One source is unavailable	Check that the expected channel is still sending and that the saved field belongs to this recording or stream. Choose the intended replacement explicitly.
Graph seems frozen	Check Pause display and the freshness indicator. Resuming display does not restart a stopped transport.

Connect the Hydrasynth through the browser

1. Power and connect the Hydrasynth. In Connections, choose MIDI device under Synth output, then press Connect MIDI device if permission or device access needs retrying.
2. Allow browser MIDI access. Check MIDI destination and select the intended instrument if more than one device matches. Do not select an unrelated MIDI output just to remove an error.
3. Use PARAM RX mode = NRPN · full engine for the full parameter catalog, and set PARAM RX on the Hydrasynth to NRPN. Choose CC · compatibility only when both sides are set for CC.
4. Leave Updates / second at 50 initially; the available choices are 25, 50 and 100. Enable PGM CHG RX on the synth only when you want the dashboard to select stored patches.
5. Open Synth mixer and press Play. The dashboard sends on MIDI channel 1. Connecting alone does not start playback or change patches.
6. If notes continue after stopping, use Connections > Stop all MIDI notes. After a device disconnect, confirm the destination and press Play again.

The browser tries the saved destination or a single identifiable Hydrasynth automatically. A changed port ID can be matched by unique name and manufacturer; multiple matches require selection. Native ALSA and browser Web MIDI are separate routes, so a successful native device list does not grant browser permission.

CHAPTER 05

Signal mixes turn data into reusable musical controls

Synth mixer and Music mixer share the same Signal mixes workflow. A signal mix is a named control value built from one or more numerical inputs. It does not mix sound. Each input is reduced to one number, normalized to a 0-100% range, and combined with the others. You then connect that result to synth parameters or music-channel effects. Begin with one input and one destination so that you can hear what each choice does.

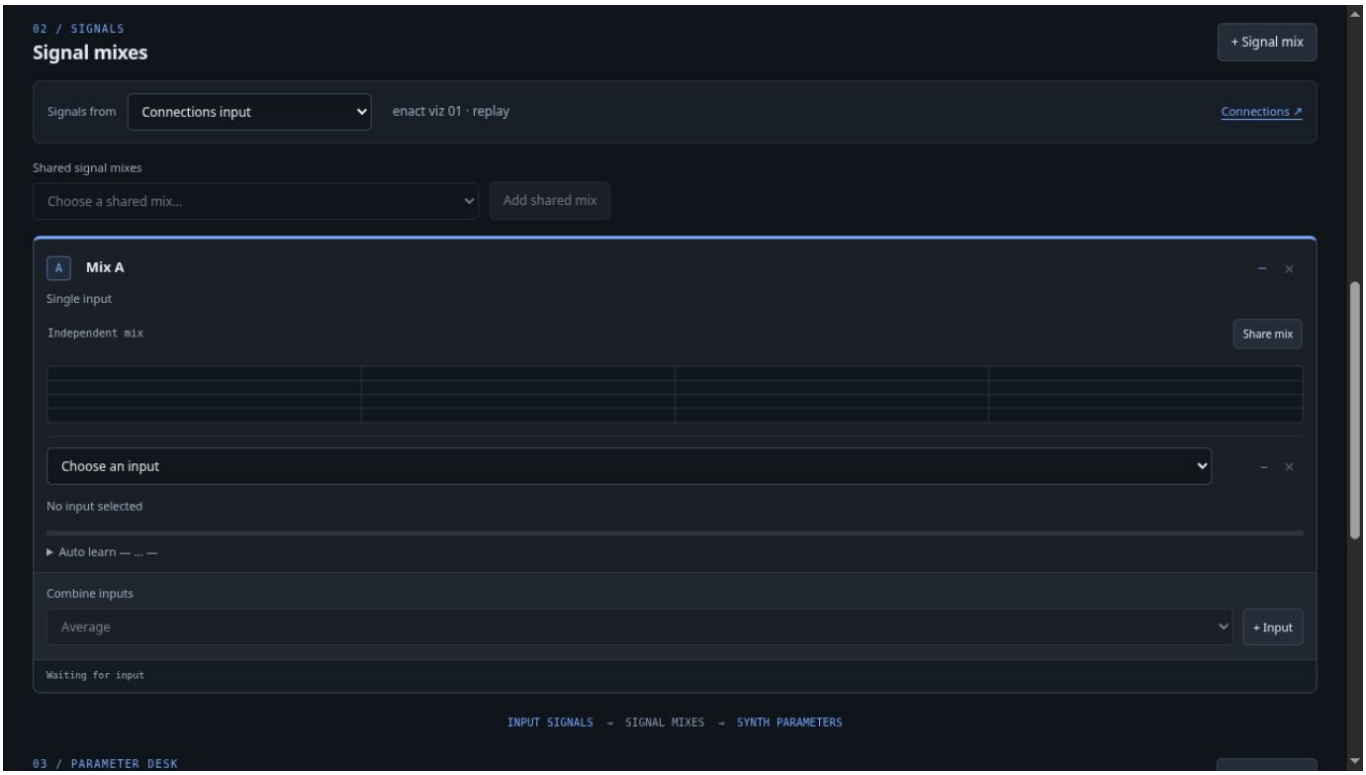


Figure 2. Signal mixes before an input is selected. Choose an input, inspect its range and then assign the mix to a parameter; the empty history here is expected while stopped.

Build and inspect a mix

1. Set Signals from. Connections input uses the recording or live stream chosen in Connections. Simulated demo signals supplies Movement, Energy and Pace for learning the controls without a sender.
2. In a Signal mix card, edit its name to describe the intended behavior, for example Arm energy or Slow brightness. Names help you identify the same control when assigning it to several destinations.
3. Choose an input. The menu groups available signals by source channel and distinguishes joint position from velocity. Check the raw numerical readout, normalized meter and recent-history graph.

4 Expand Auto learn or Fixed range beneath the input. Choose Read as if the field supports several reductions. Leave a scalar joint on its normal direct reading; select an array reduction deliberately.

5 Press + Input to add another source to this mix, then choose Combine inputs. A mix supports up to six inputs. Press + Signal mix to make another independent control; either mixer supports up to four mixes.

6 Assign the mix to a parameter or channel effect. A mix with no destination can be inspected without changing the sound.

Changing Signals from replaces the mixes' input selections with fields appropriate to the new mode and detaches shared links. Save a session before switching modes if you want to preserve a carefully prepared mapping.

Read as	Use and interpretation
Direct value	Use one scalar reading, such as a labelled joint position or velocity.
Average	Use the mean of an array. Positive and negative entries can cancel each other.
RMS magnitude	Use the size of an array's values without positive and negative cancellation; useful when magnitude is more meaningful than direction.
Lowest value / Highest value	Follow the most extreme valid value in the selected array.
One element	Read one flattened array element. Element index starts at 0: index 0 is the first element. Keep the sender's array layout consistent.

Calibrate input ranges

Auto learn expands the observed minimum and maximum as usable values arrive. For recordings, available saved bounds help establish the range; for live input, the app learns from fresh frames. A constant learned signal maps to the midpoint rather than causing a division error. Early values may therefore have a large effect until the performer has explored a representative range. Let the input cover its normal movement before deciding whether the output sweep is suitable.

1 Leave Range on Auto learn while observing a representative movement or recording segment.

2 Inspect the displayed bounds and normalized meter. Confirm that quiet and active movement occupy different parts of the range.

3 Press Freeze range to retain the learned bounds as a fixed calibration. This helps repeat a mapping consistently during a session.

4 For known units, choose Range = Fixed and enter Minimum and Maximum yourself. Use a finite minimum smaller than the maximum. Values beyond the bounds clamp at the corresponding end of the normalized range.

5 Press Learn again when the input scale changes and you want to replace the calibration. This is specific to the chosen input; it is not a reset of every mix.

Combine several inputs

Combine inputs	Result
Average	Equal influence from each normalized input. A useful starting choice for several joints.
Weighted blend	A weighted average. Expand each input to set its Weight. For example, weights 3 and 1 give the first input three times the influence of the second.
Difference	Compares the first input against the average of the remaining inputs. Equal normalized values produce the midpoint; a larger first input moves upward and a smaller first input moves downward.
Lowest	Follows the smallest normalized input, so all inputs need to rise for the result to become high.
Highest	Follows the largest normalized input, so one active input can produce a high result.

Inputs are normalized before combination, so fields with very different raw scales can contribute together. A required missing input makes the mix unavailable. Remove unintended sources or choose replacements explicitly; the other inputs do not hide their absence.

Share definitions between the two mixers

1. On a prepared mix, press Share mix. It is now a shared definition in this browser.
2. Open the other mixer. Use the same Signals from mode, choose the definition under Shared signal mixes, then press Add shared mix.
3. Assign local destinations in the second mixer. The shared definition supplies the inputs and combination, while each mixer keeps its own synth parameters or stem-effect mappings.
4. When you switch between mixers, edits to shared input definitions, combination and fixed bounds are synchronized. Automatically learned ranges stay local.
5. Press Make independent if you want to branch the mix without subsequent shared edits. The current definition becomes an independent local copy.

Control the amount of influence

Signal strength controls each mapping’s influence. At 0%, the manual/base setting applies and the mapping does not wait for input. At 100%, the full mapped range applies. Intermediate values blend toward the base. Try 25-50% for a dramatic effect, then adjust From and To. Music frequency controls use logarithmic blending.

CHAPTER 06

Synth mixer play and shape browser or Hydrasynth sound

Synth mixer has three areas: Shape your sound, Signal mixes, and Synth parameters. It creates notes and parameter changes locally. Browser output needs no provider key or generated track. MIDI output uses the destination selected in Connections. The small output label beside Shape your sound identifies the current route, so check it before troubleshooting silence.

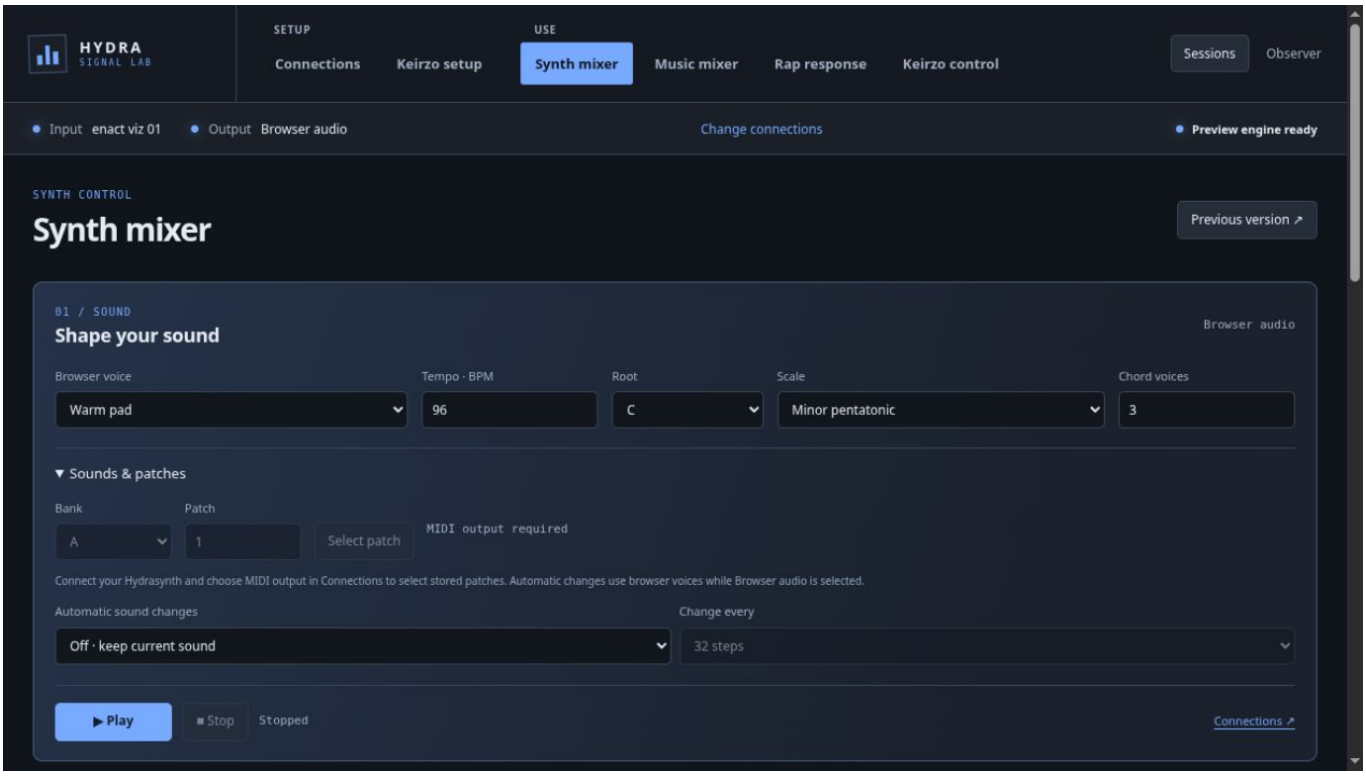


Figure 3. Synth mixer places the sound and transport controls above the named input mixes. Browser voices work without connected hardware.

First sound without hardware

1. In Connections, select Browser audio and confirm Test sound works. Open Use > Synth mixer.
2. Choose Browser voice = Pluck or Warm pad. Set Tempo · BPM to 96, Root to C and Scale to Minor pentatonic. Set Chord voices to 1 for a simple line or 3 for chords.
3. Leave Automatic sound changes at Off · keep current sound. This keeps the timbre stable while you learn the mapping controls.
4. Press Play. Existing manual parameters can produce sound without an active movement mapping. If a saved mapping is waiting for input, choose its correct source, set its Signal strength to 0%, or temporarily disable that parameter.

5 Choose Simulated demo signals under Signals from. Name a mix Movement brightness and select Movement. In the Filter cutoff card, choose that mix under Signal mix.

6 Listen while reducing Signal strength or changing the mapped range. Press Stop when finished. Save a named session if you want to preserve this experiment.

Browser voice offers Glass, Pluck, Warm pad, Reed, Bell, Soft bass and Custom. These browser recipes are separate from Hydrasynth factory patches. Tempo accepts 20-240 BPM and Chord voices accepts 1-8. Root and Scale constrain the musical pitches.

Use the parameter desk

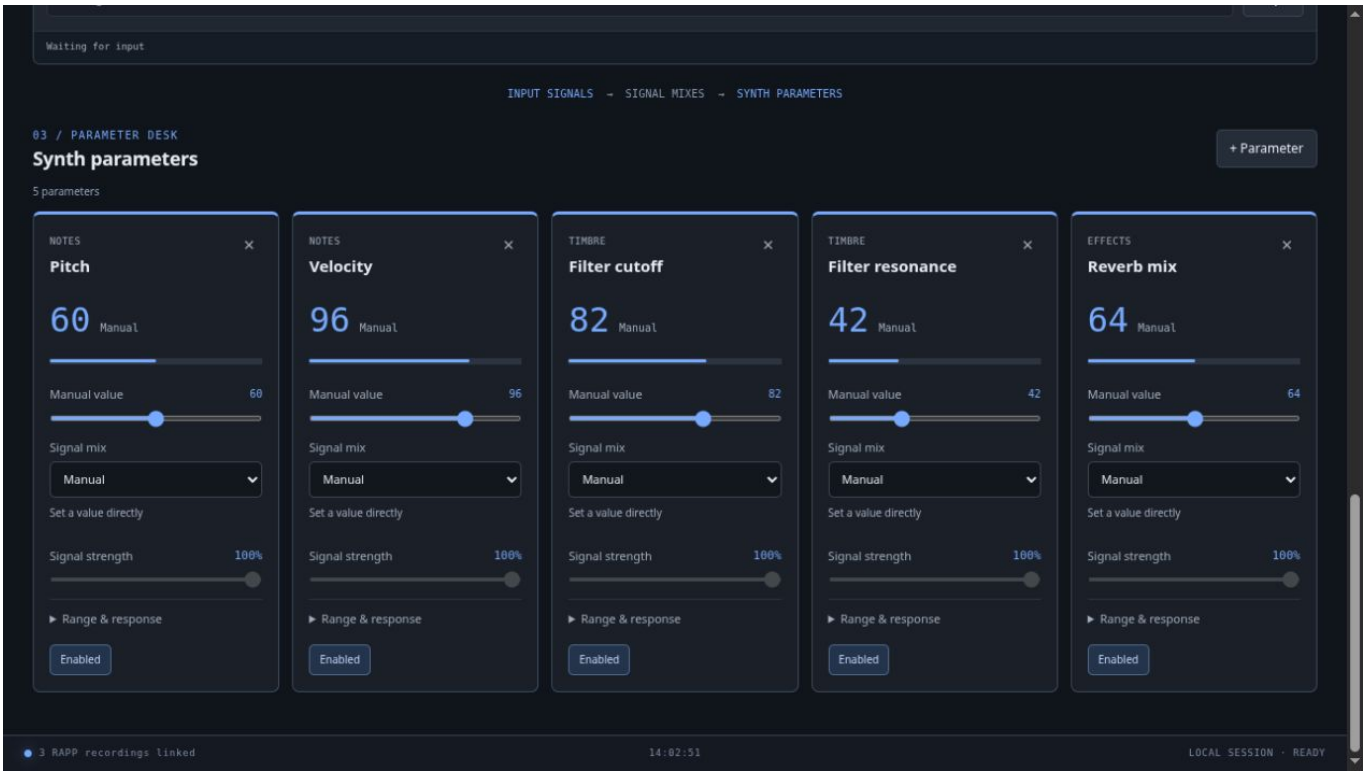


Figure 4. Parameter cards show the effective value, a manual base, the selected signal mix, Signal strength, and the expandable mapping range.

1 Find the desired existing card, or press + Parameter. Search in Find a parameter or narrow Group, then select the destination to add it.

2 Choose Manual in Signal mix to set the value directly. For an enum control, use its named choices; for a continuous control, use its slider.

3 Select a named mix to map the parameter. The manual slider becomes Base value, used at 0% Signal strength and as the reference for partial strength.

4 Open Range & response. Set From and To to the desired musical bounds. Start with a modest range, then widen it while listening.

5 Choose Response and optionally Reverse. Check the large effective-value readout and meter while the source moves.

6 Use Enabled/Disabled to retain a card but stop that parameter from contributing. Use the × button only when you want to remove the card.

The desk contains note, harmony, rhythm, timbre and effect destinations as well as the published Hydrasynth catalog. Hardware controls carry MIDI labeling. A MIDI only or unavailable reason means the current output cannot send that control; it is not evidence of a broken input. Hardware mappings remain saved when you switch to browser audio. Full-engine NRPN entries need NRPN selected in Connections. Reference catalog entries can include documented ambiguities; the interface does not invent a sendable address for unsupported definitions.

Response	Practical effect
Linear	Even progress from From to To as the normalized signal rises.
Smooth	Gentler movement near each end of the range.
Exponential	Less movement at the lower end, increasingly strong movement as the signal rises.
Inverse	Reverses the normalized signal. Reverse also changes direction, so avoid applying both unless that is intentional.
Steps	Quantizes the normalized signal into five levels. Useful for distinct changes rather than a continuous sweep.
Threshold	Switches between the two ends at the midpoint. Useful for an on/off-style musical change.

Build rhythm deliberately

Use + Parameter for Hits per 16 steps, Pattern rotation · steps, Note repeats · per step, Swing · %, Accent strength · %, Gate length · %, Note chance · %, or Tempo · BPM. The clock generates a repeating step pattern from the data; it does not detect a prerecorded beat inside a movement file. Fewer hits and lower note chance create more space. More repeats subdivide selected hits. Gate changes how long a note is held, while velocity changes its attack level.

For a clear demonstration, map one movement mix to Filter cutoff first. Then add Hits per 16 steps with From = 4 and To = 12. Keep pitch and tempo manual while listening to these two changes. If both mappings feel too active, lower each Signal strength separately. Reusing one mix creates a relationship between brightness and density, but their ranges and amounts remain independent.

Choose Hydrasynth patches

1. Connect MIDI and match PARAM RX as described in Connections. On the instrument, enable PGM CHG RX if patch selection is required.
2. Expand Sounds & patches. Choose Bank A-H and Patch 1-128, then press Select patch. These are the front-panel bank and one-based patch locations.
3. Listen to the selected patch before mapping many parameters. Patch selection replaces unsaved edits in the synth’s current edit buffer.
4. For changing sounds during playback, choose Cycle, Random or Follow mapping under Automatic sound changes. Choose Change every from 16, 32, 64 or 128 steps.

5 For a limited MIDI palette, choose Custom selection under Patch palette and enter locations such as A001, B017, H128. Verify each location contains an appropriate sound on this instrument.

6 For Follow mapping, add Sound palette position · % and map it to a mix. Return Automatic sound changes to Off when you need a stable patch.

Browser automatic sound changes select browser voices; MIDI changes select stored patches. Playback can continue after a brief patch transition. Parameter edits affect the current synth buffer rather than storing a patch permanently.

Understand stopped and waiting states

Switching dashboard workspaces keeps the Synth transport and parameter processing running. Stop is the explicit end of playback. Missing data in an enabled, available mapping above 0% strength makes the synth wait rather than substituting another source. Check which card is mapped, whether its mix has valid inputs and whether the output is MIDI or Browser audio. If a saved hardware mapping is unavailable in browser mode, it stays saved without pretending to affect the browser sound.

Previous version opens the older Synth mapping-tab interface and its independent settings. Loading sessions or changing connections can stop playback; verify the restored output and source before restarting.

CHAPTER 07

Music mixer create instrumental tracks and shape their stems

Music mixer plays generated audio through the browser. A track is the musical recording; stems are separated instrument layers that can be adjusted independently. The workspace filters vocal stems out of its instrumental playback and mappings. A fresh Git clone provides movement recordings and voice demos, but does not automatically include music you generated on another laptop. New music and stem separation require an API key, internet access and provider credits; adjusting or replaying already prepared local audio does not generate a new performance by itself.

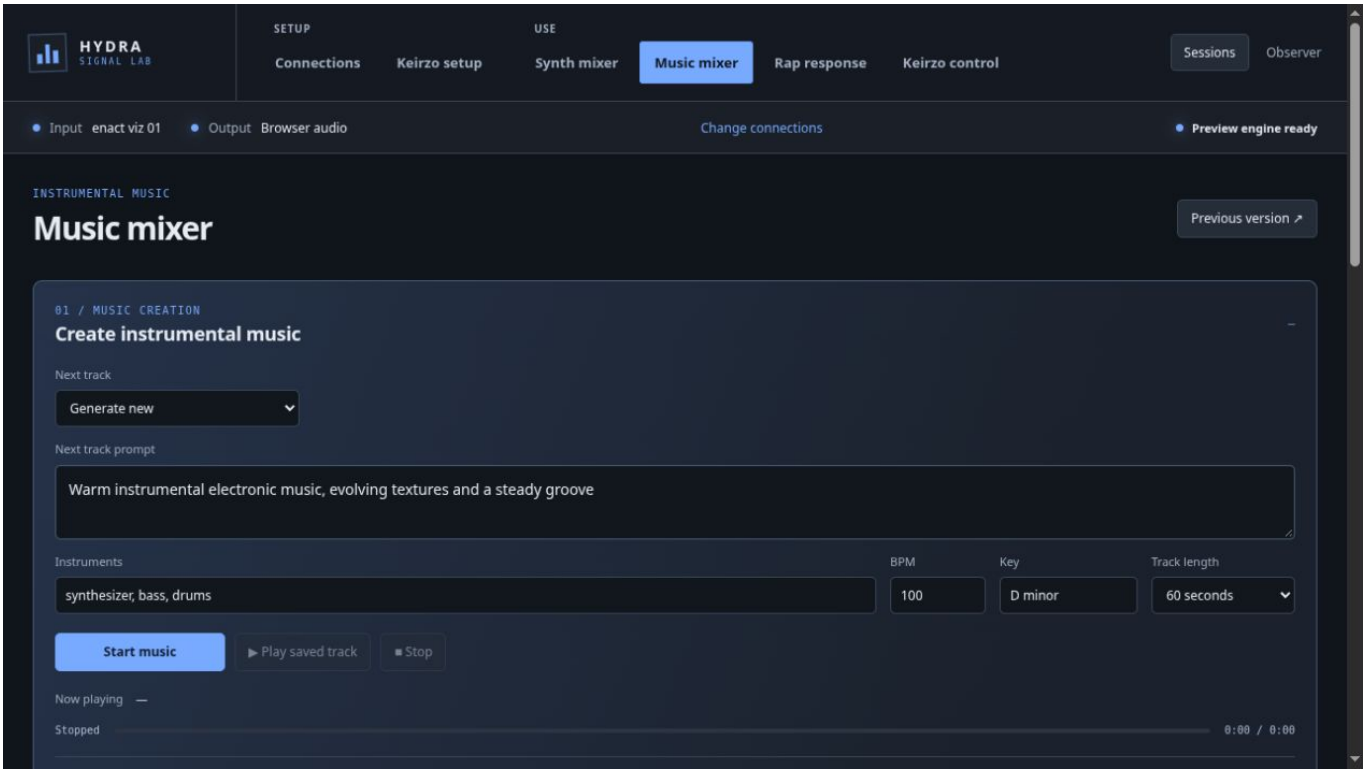


Figure 5. Music mixer combines creation controls, reusable signal mixes and a desk for the master and instrumental stems. Saved tracks depend on audio retained on the local server.

Play existing music without automatic generation

1. Open Use > Music mixer. Set Next track to Loop before starting if you want only the current track to repeat.
2. Expand Recent tracks and choose a completed saved track. Press Refresh if a newly completed track is missing from the list.
3. Press Play saved track. Check the selected title and Playing status. The current track repeats in Loop mode without requesting a replacement.
4. Adjust the Master level, then the available stems. Scroll the mixing desk horizontally if some channels are outside the visible area.
5. Press Stop when done. Merely switching to another workspace leaves this transport running.

If no track is available, generate one or transfer the intended local music library. Sessions JSON contains references, not audio. Current Music mixer playback requires prepared instrumental stems. For an older track with missing stems, use the Previous version interface’s preparation workflow, which uses credits, or choose a fully prepared track.

Generate a first instrumental track

1. Confirm the backend has its API key and FFmpeg setup. Address any setup message before generating.
2. In Create instrumental music, enter Next track prompt. A useful example is: Gentle instrumental electronic groove, spacious chords, clear bass and restrained percussion.

3 Set BPM, Key, Instruments and Track length. The length choices are 20, 40 or 60 seconds. For example, try 96 BPM, C minor, and synthesizer, bass, drums. Keep these fields consistent with the prompt.

4 Choose Next track deliberately. Loop creates a starting track if none is selected and then repeats it. Generate new and Generate similar enable continuing replacement generation after the starting track.

5 Press Start music, or Create starting track when that is the displayed action. Wait through music generation and stem preparation. The controls report the current stage rather than immediately presenting unfinished audio.

6 When playback begins, listen before adding mappings. Use Stop to end playback and prevent future automatic requests. Work already submitted can still finish and use credits.

Generated BPM, key and style are musical requests, not guaranteed measurements. Beats can drift and stems can contain bleed. Listen before synchronizing a robot or rap vocal.

Next track	Behavior while playing
Loop	Repeats the current track. No replacement generation is requested.
Generate similar	Prepares another track from the prompt and, when available, a low-strength reference to the current audio.
Generate new	Prepares a fresh interpretation of the prompt. This is the default continuing-generation choice.
Choose old	Select Saved track for the next transition. The chosen old track then repeats until you choose another. This choice does not request new music or stem separation.

Generate new and Generate similar keep one spare track ready, including stems. When a transition consumes that spare, another can be requested; there is no session generation cap in this mixer. Editing the prompt, BPM, key, instruments or length while continuing generation is enabled updates the waiting request after a short typing pause. Keep an eye on Next track if several laptops share one provider account.

Up next describes pending music. Ready replacements crossfade near the loop boundary. If preparation is late, the current track repeats. If generation fails, it also keeps repeating until Retry next track succeeds. A temporary rejection because another music or stem job is active can retry after a delay, while failed or uncertain provider outcomes are not automatically resubmitted. Switching to Loop or Choose old stops further generation requests; already submitted work can finish and appear in Recent tracks. A crossfade already in progress completes before the following choice applies.

Mix manually first

Control	Use
Volume fader	Sets the channel’s manual level. Master controls the combined output.
Mute	Silences a channel regardless of its mapped volume. Press again to restore it.
Solo	Available on stems. Isolates selected stems for listening and takes priority over mappings.
Pan	Places a channel between left and right.
Low pass / High pass	Reduce high or low frequencies respectively. Excessive filtering can make an otherwise playing channel nearly inaudible.
Delay / Reverb	Add repeats or a reverberant tail. These can remain audible briefly after a musical event.
Pitch shift	Shifts pitch from -12 to +12 semitones without changing duration. Master and stem shifts add together.
Playback speed	Master only, under More effects. Changes speed from 75% to 125% while preserving pitch and keeping the stems together.

Vertical meters show effective volume settings, not measured loudness. A high meter does not prove that the file is sounding. Local pitch/speed processing adds a short delay, and large pitch shifts can alter transients.

Add movement-driven effects

1. Under Signal mixes, choose Connections input or Simulated demo signals. Prepare a named mix using the input and calibration workflow described earlier.
2. On the intended channel, press + Map a signal. Choose Signal mix, then the effect under Controls.
3. Open Range & direction. For a first pan experiment, choose a moderate span such as From = -0.5 and To = 0.5. Use Reverse if movement should travel the other way.
4. Adjust Signal strength while listening. At 0%, the channel keeps its manual setting. At 100%, it follows the full mapped range.
5. Reuse the same mix on another channel or another effect. For example, let movement pan a percussion stem while gently opening the master low-pass filter.
6. Check mute, solo and manual levels when a volume mapping seems ineffective. Mapped volume multiplies the fader, so a fader at zero remains silent.

An audible channel waits for the valid inputs needed by its active mappings. Missing inputs on muted or solo-excluded stems do not silence the other audible stems. If the transport says Waiting for mapped input, inspect the relevant mix, its fields, and the source mode; reduce that route to 0%

or remove it when comparing with manual playback. Preferences survive Stop and reload, but reloading does not start audio or arm new generation.

CHAPTER 08

Rap response record type perform and reuse replies

Rap response turns a short message into a rhyming answer, or performs words you supply. It supports a generated full mix and an isolated voice over a playing mixer. Open Use > Rap response. The six fixed voice choices are Eva, Cold Android, Peter, Elowen, Maisie and Warren. A voice identity is consistent, while delivery and expression can vary between generated performances.

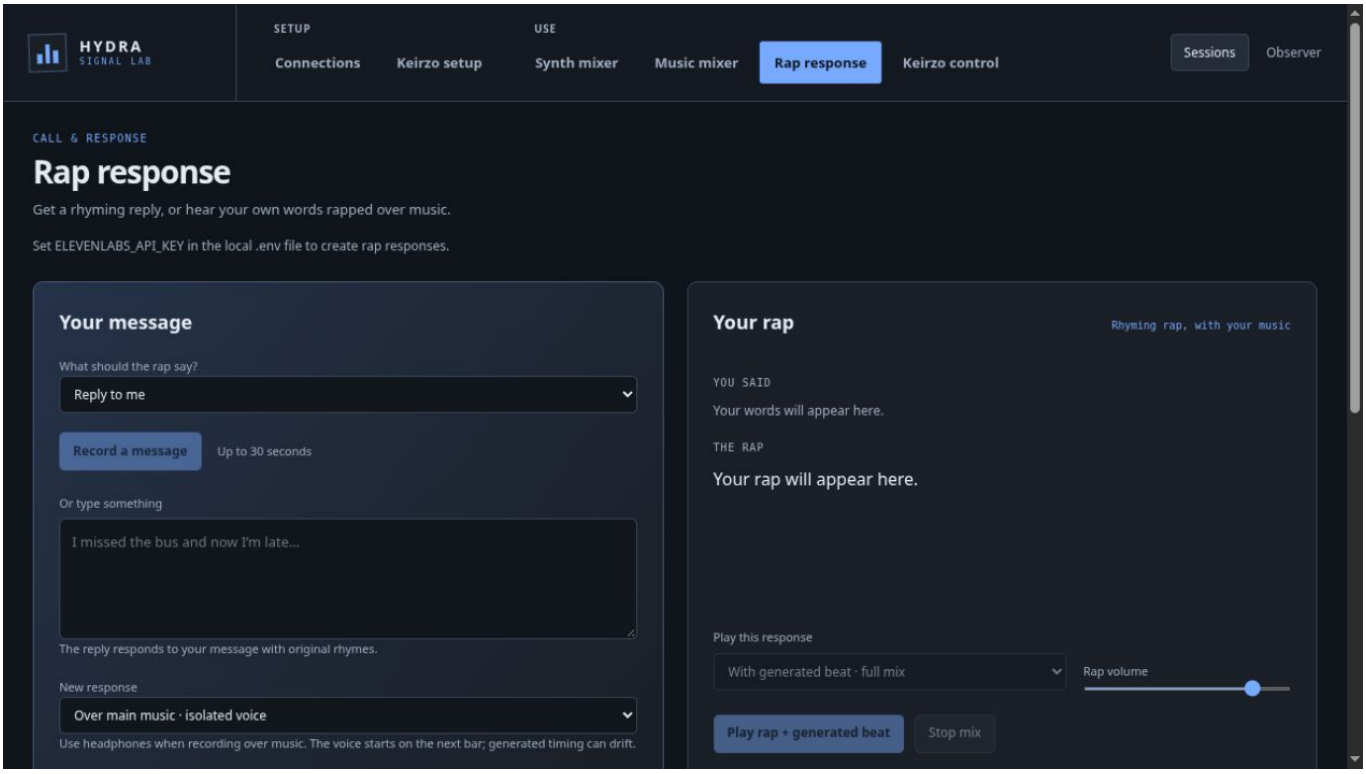


Figure 6. Rap response keeps message creation on the left and the transcript, lyrics and playback on the right. Voice demos at the bottom can be auditioned without generation.

Audition voices without using credits

1. Expand Voice demos at the bottom of the workspace.
2. Press Play Eva, Play Cold Android or another named button. Only one demo plays at a time.
3. Use its Stop button, choose a different demo, or close Voice demos to end the preview.
4. Select the identity you want under Voice for new raps. Listening to a demo does not change that selection automatically.

The demos are bundled and need no key. They use clean audio, without your Pitch shift or Autotune settings. New delivery and timing can differ.

Create a typed reply

1. Set What should the rap say? to Reply to me.
2. Type a concise message in Or type something. For example: I finally finished my project and want to celebrate with a calm victory song.
3. Choose Voice for new raps, Beat style and Length. Beat style offers Boom bap, Trap and Lo-fi; Length offers 4 or 8 bars.
4. Choose New response. Use With generated beat · full mix for a standalone result, or Over main music · isolated voice when you already have a backing mixer playing.
5. Set Requested BPM between 70 and 140. If a valid backing is playing in isolated-voice mode, its tempo is used for the new rap and must be within that range.
6. Choose whether to enable Play when my new response is ready. Press Make rap response once and follow the status through writing, performing, separating, converting and local effects.
7. When Rap ready appears, inspect You said and The rap, choose the desired Play this response mode, and press its playback button if it did not auto-play.

Record a message

1. Choose Reply to me and set the desired voice, style, length and output mode before starting. These settings are captured for the request.
2. Use headphones if music is already playing. This reduces backing audio entering the microphone.
3. Press Record a message and grant browser microphone access. Speak a clear, short message. The maximum recording is 30 seconds.
4. Press Finish & rap when finished. At the limit, recording finishes automatically. This begins transcription and the rap workflow; it is not merely a recording preview button.
5. Watch the status and resulting transcript. Use Cancel if you need to abandon the remaining work. Leaving Rap response also releases the microphone and stops this workspace’s playback.
6. If the transcript is unsuitable, use typed input for a new request with the intended message. A previously submitted generation is separate and may still have used credits.

Microphone transcription runs locally with Whisper Small after its model has been installed. The original microphone recording is not sent to ElevenLabs. The transcript or supplied lyrics are sent for the requested provider workflow, and generated vocal audio is sent for voice conversion. New raps use provider services and credits even though transcription itself is local. The study recorder retains microphone attempts, including canceled attempts, when study recording is enabled; canceling generation is not a request to delete study data. The generated full mix is also sent to the provider for stem separation.

If local transcription is unavailable, the record control explains the missing setup and typed input remains available when the generation service is configured. If microphone access is unavailable or denied, check the browser’s site permission and Linux input-device selection. Check the displayed backend setup issues separately: giving microphone permission does not configure an API key.

Perform your own lyrics

1. Set What should the rap say? to Rap my words. The recording row is hidden and the text field becomes Words to rap.
2. Enter the words exactly as you want them supplied to the performance. Preserve line breaks for your intended lyrical grouping.
3. Keep within 48 words for 4 bars or 96 words for 8 bars. The on-screen counter shows your current allowance.
4. Set the voice, effects, beat and playback choices, then press Rap my words. This skips writing a reply and submits your text as the lyrics.

Play over a mixer or with the generated beat

For Over main music, first start Music mixer or Synth mixer, then navigate to Rap response. Navigation keeps the backing running. The isolated vocal is scheduled at the next bar and uses the backing’s audio clock. If the backing is stopped, invalid or changing speed, wait for a stable transport and press Play again. A generated performance can drift from the backing even though its start is aligned. Changing or stopping the backing stops the vocal so that it does not continue against an unrelated clock.

With generated beat plays the processed voice and its own generated instrumental. Full-mix microphone recording and completed full-mix playback stop other dashboard audio; submitting a typed generation alone does not. Play this response controls how a completed result is heard; New response controls the next request. Use Rap volume for listening balance, Stop voice or Stop mix to stop the result, and Download full mix or Download voice for available audio files.

Play when my new response is ready applies to the request you just made. Opening Saved responses or loading a session never records, generates or starts a saved rap automatically. Leaving this workspace stops its voice/demo playback and cancels pending auto-play, while a job already submitted to the server may continue in the background. This is the deliberate exception to the mixers’ persistent playback during ordinary workspace navigation.

Customize a voice locally

Open Customize the voice. Pitch shift spans -12 to +12 semitones. Autotune begins at 0% for clean vocals; choose its Key and Scale, then use Melody = Follow the voice for pitch correction or One note, Two notes, Rising, Falling, Arpeggio or Hook for a repeating pitch pattern. Melody octave offers 2-5, and Note length offers half a beat, one beat or two beats. Pitch shift transposes the processed vocal; the instrumental retains its timing and pitch.

1. Select a completed rap under Saved responses. Open Customize the voice and set a modest Pitch shift or Autotune amount.
2. Press Apply to this rap when enabled. The app uses the retained clean vocal to create another local version without another provider generation request.
3. Compare the resulting version with the original. Reset effects returns the controls to clean settings for a subsequent application or request.
4. Generate a new rap if you want a different voice identity or different words. Apply to this rap preserves the source recording’s identity and lyrics.

Recover or cancel work carefully

Do not repeatedly submit a request because generation takes time. Recover response checks the existing request identity after interruption. Cancel stops remaining work but cannot undo a paid call already in flight. Failed voice conversion is reported without substituting another voice. Refresh Saved responses to locate completed work.

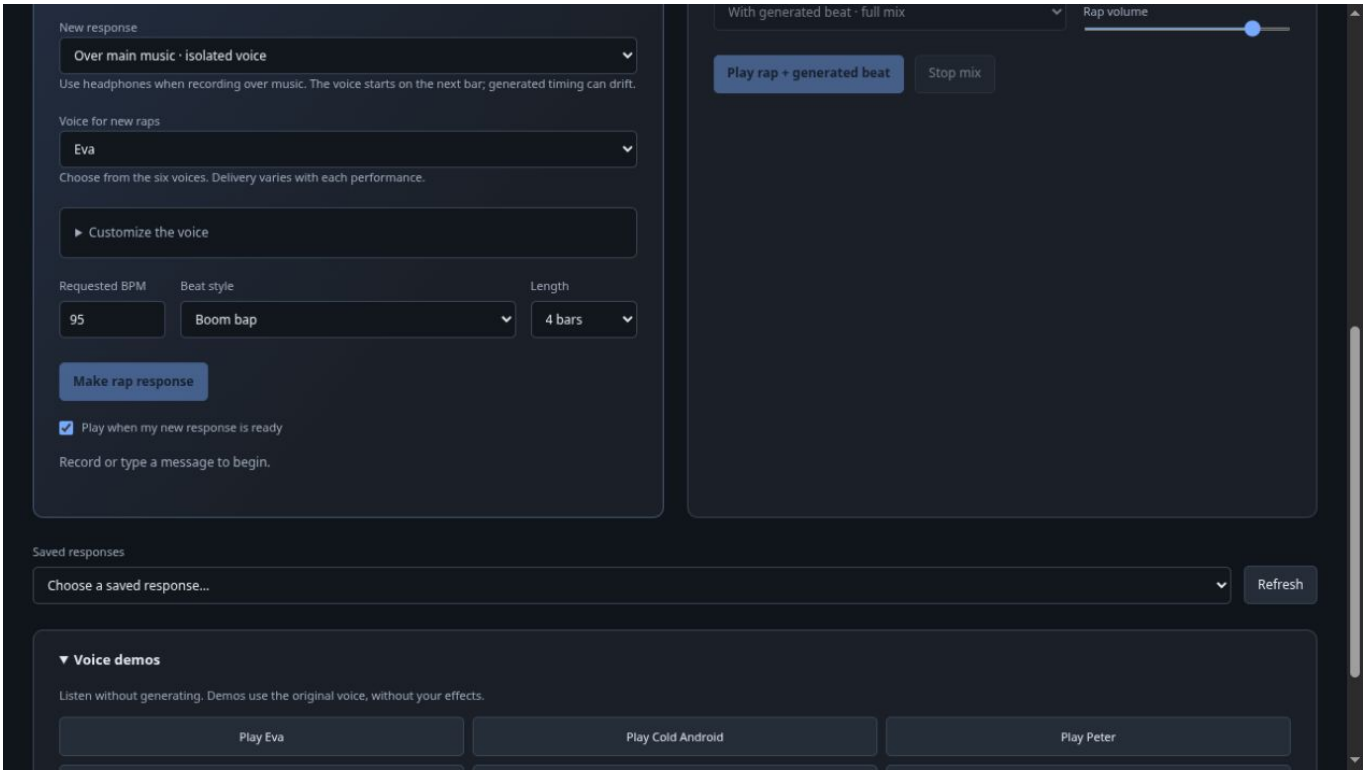


Figure 7. The expanded Voice demos area provides six bundled examples. The screenshot shows stopped demos and an empty response; no provider generation has been run.

CHAPTER 09

Saved sessions presets and moving a setup

Sessions is a named settings library opened from the top bar. It captures the controls across workspaces so you can return to an arrangement or distribute the same starting configuration to several laptops. It is distinct from the academic study’s participant sessions and recorded events. Saving a musical session does not label a new participant, export study data or save a new audio recording.

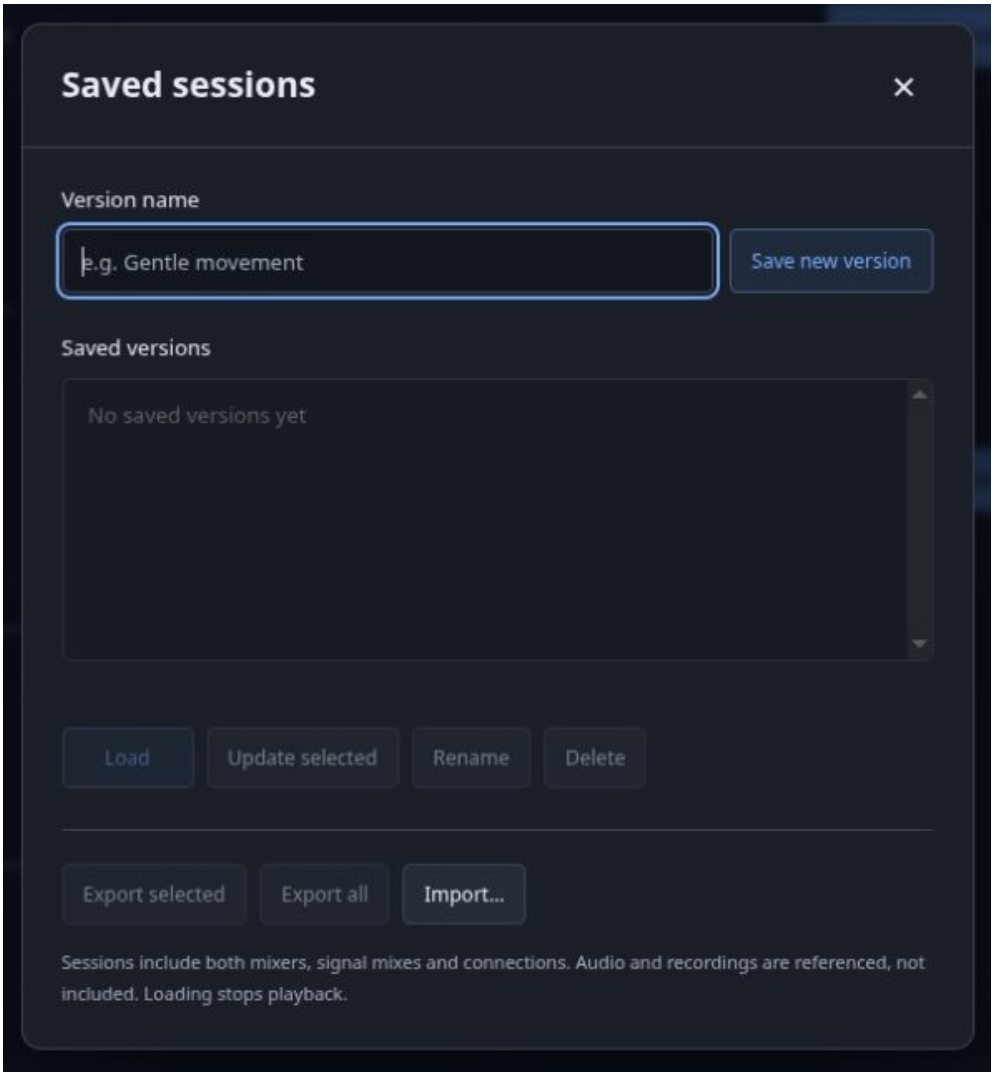


Figure 8. Saved sessions holds named versions of dashboard preferences. Export selected or Export all produces a portable settings JSON file.

Save branch and restore a version

1. Prepare the Connections choices, mixer mappings, sound settings and other controls you want to retain. Give signal mixes clear names before saving.
2. Open Sessions, enter Version name and press Save new version. Use a descriptive name such as Workshop baseline - browser - slow brightness.
3. Make an experiment. Save new version again with another name to preserve both versions. Use Update selected only when you intend to replace the selected version’s snapshot.
4. Choose a version under Saved versions to inspect its details. Rename changes its name; Delete removes that saved settings version.
5. Press Load to restore. The app stops playback and robot/live activity, reloads the controls, and does not automatically reconnect live movement or start playback. The normal browser MIDI discovery and permission flow may run after reload.
6. Check the recording or live address, audio/MIDI output, available tracks and hardware state. Reconnect live data or MIDI where needed, then press Play explicitly.

Sessions retain mixer controls and mappings, shared definitions, prompts and asset IDs, connection preferences, Rap response and robot/drum preferences, and previous-interface settings. Running jobs remain separate. Restoration neither repeats a paid request nor verifies a physical instrument’s state.

Export a configuration for six laptops

1. On the source laptop, choose a tested baseline under Saved versions. Press Export selected for one version or Export all for the full settings library.
2. Keep the downloaded JSON with the workshop setup files. Give the file a version/date in its filename so operators can identify the intended release.
3. On another laptop, start its own dashboard and open Sessions > Import.... Select the exported JSON.
4. Select the imported version and press Load. Verify the local input/output and confirm every referenced recording or generated track exists on that laptop.
5. Use a new local version name for laptop-specific changes, such as its MIDI destination or workshop live-stream address. Export that variant if it needs to be retained outside the browser.

The portable JSON contains settings and references, not generated audio, microphone recordings, the API key, model files or the robot’s head-calibration file. Do not treat it as a complete installation backup. Bundled movement recordings normally exist in every clone, whereas tracks in var/elevenlabs and raps in var/rap-response are local server assets. Copy the intended assets separately when those references must resolve on another computer; do not indiscriminately clone one laptop’s participant study identity.

Browser storage and recovery

Preferences and named versions belong to this browser profile and page origin. localhost:8790 and 127.0.0.1:8790 are different origins. Clearing site data or changing profiles can make them unavailable. Use one address consistently and export important versions.

The library supports 50 versions and validates imports. If storage is full or unavailable, keep the page open, export existing versions, allow site storage or remove older versions, then retry. Import a Sessions file, not an older preset-library file.

Use older presets only in their matching interface

The older Synth interface has mapping tabs and its own Save preset, Open in new tab, Export and Import controls. Its preset library is different from Sessions JSON. Previous version in Music mixer likewise retains an older workflow. Use Sessions for a complete new workshop baseline.

CHAPTER 10

Observer inspect telemetry and diagnose behavior

Open the read-only Observer from the top bar or http://127.0.0.1:8790/observer/. It helps distinguish a mapping producing values from a later audio/hardware problem. It is not a microphone meter and cannot confirm audible sound or physical robot contact.

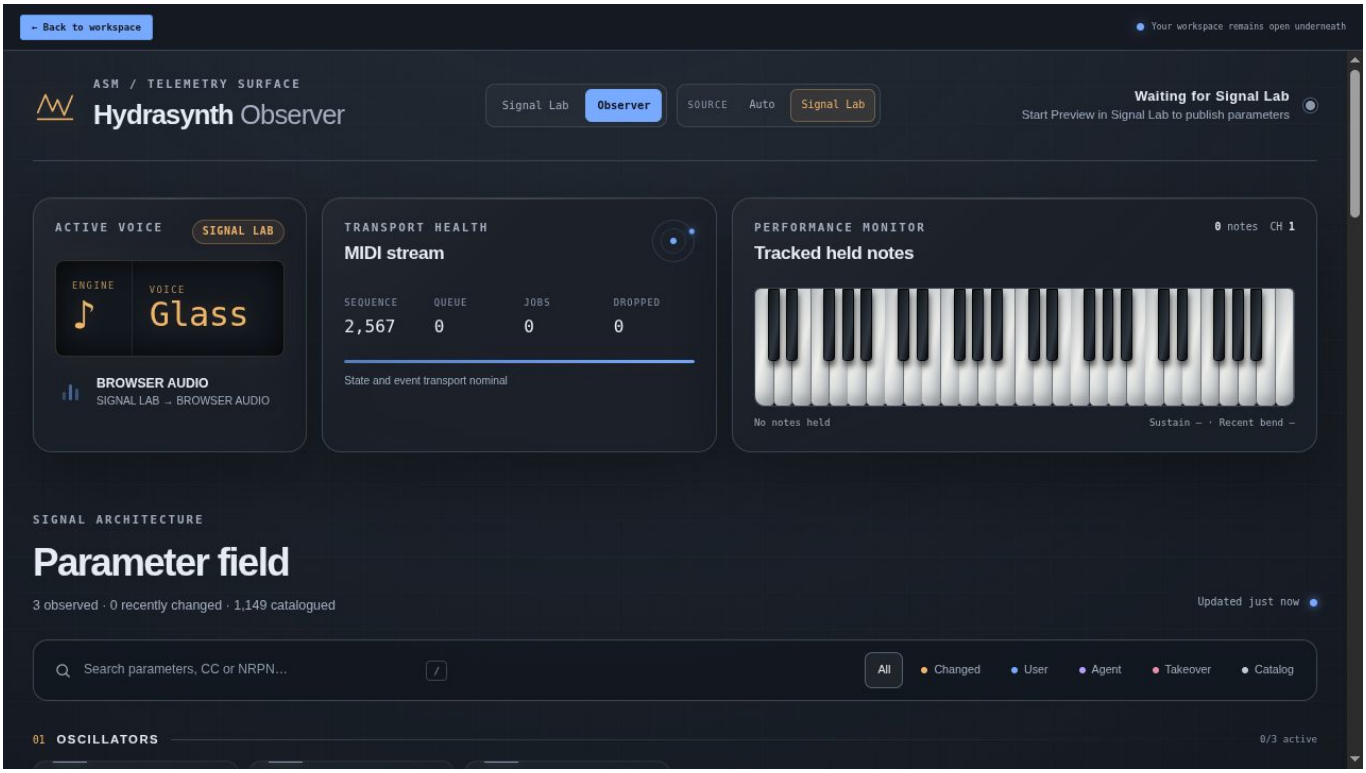


Figure 9. Observer exposes the selected source, transport health, performance state, parameter field and event timeline. Demo data is labeled and must not be mistaken for a connected instrument.

Choose the source intentionally

Source = Auto follows active Signal Lab preview telemetry and falls back to native/demo telemetry when the preview stops. Source = Signal Lab stays locked to the dashboard’s sonification feed. This feed can describe browser-audio playback as well as MIDI commands, so it does not require the Hydrasynth to echo every sent value. In a no-hardware demonstration, the fallback can be labeled Demo data. That display is an interface demonstration, not evidence of a connected synth.

1. Start a simple Synth mixer performance with one mapped parameter.
2. Open Observer through the top bar so you can return to the workspace without intentionally stopping playback.
3. Choose Signal Lab if you want to watch only the browser performance. Check the connection label and route before interpreting values.

4 Search for your mapped parameter. Watch its value and short history while the input changes.

5 Return to the workspace and press Stop. In Auto, observe the source fallback; in Signal Lab, expect a waiting/inactive state rather than a new performance.

Read the panels

Panel or readout	Interpretation
Active voice and route	Shows the known patch/voice and data route. An unknown patch is not proof that patch selection failed; native tracking depends on the messages the synth has transmitted.
Sequence	A counter for observed event ordering. Gaps can indicate missed notifications.
Queue / Jobs / Dropped	Native transport health. Queued events and active jobs should settle after completed work. Rising dropped counts mean the observed history is incomplete.
Tracked held notes	Notes known to the service or feed, plus channel and expression readouts. This is tracked state, not acoustic pitch detection.
Parameter field	Observed or reference parameters, their values, provenance, histories and any takeover windows.
Activity timeline	Recent incoming and scheduled events. Use it to relate a control change to a note, parameter event or job transition.
Snapshot age	How recently a snapshot reached the page. An old snapshot can explain why the display appears frozen.

Filter parameters and understand provenance

Search parameters, CC or NRPN by name or number. The slash shortcut focuses the search box. All shows the available observed view; Changed emphasizes recent changes; User shows front-panel or external MIDI input; Agent shows generated control and automation; Takeover shows currently protected controls. Catalog adds the published reference definitions even when a value has never been observed. A catalog value is reference information, not a live measurement.

Who moved it? distinguishes User, Agent and Agent echo. An echo is an incoming value recognized as matching a recent outgoing command, rather than a new front-panel gesture. The native service normally gives a user-moved CC or NRPN parameter a short takeover window, three seconds by default. Conflicting native automation can be blocked until that window expires. Use this interpretation for native-service telemetry; it is not a claim that every independent Web MIDI sender participates in the same arbitration.

Diagnose a performance in order

1. Confirm the source badge. A lively demo feed does not explain silence from a stopped real transport.
2. Check snapshot freshness and any connection error. If the page cannot reach the local server, inspect the terminal running the dashboard.
3. Check whether notes or parameter events appear when expected. If they do not, return to the mixer’s transport, input freshness, mapped destination and enabled state.
4. If values change but browser sound is silent, check Test sound, mute/solo, faders, filters and Linux output routing.
5. If MIDI values are sent but the instrument does not respond, check destination, channel, PARAM RX and the instrument’s audio path. For patch tracking, also check its transmitted bank/program settings.
6. After an unplugged native MIDI device, reconnect it and restart the native dashboard/service as described in the Linux instructions. The native backend reports loss and stops rather than silently switching to another device.

When documenting a fault, record the source mode, route, snapshot age, parameter, error and action taken. A screenshot preserves the visible state; study records provide the fuller history. The bounded on-screen timeline is not a workshop archive.

CHAPTER 11

Keirzo hardware connections and first checks

Keirzo setup is the hardware test bench. Open Setup > Keirzo setup, or http://127.0.0.1:8790/#robot-setup. Use it when first connecting a robot, transferring a calibration, testing a part, or investigating a connection fault. Keirzo control is the everyday performance view. Once the rig and saved head range are verified, it can prepare the required hardware when you press Start robot.

Demo mode is not a robot simulator or a motor-output lock. make dashboard-sonify and make dashboard-demo simulate the native Hydrasynth telemetry service, but keep the real robot controls and browser Web MIDI available. A connected robot can move when you press its torque, test, or start controls. Simply opening the dashboard does not connect motors, enable their torque, or start a performance.

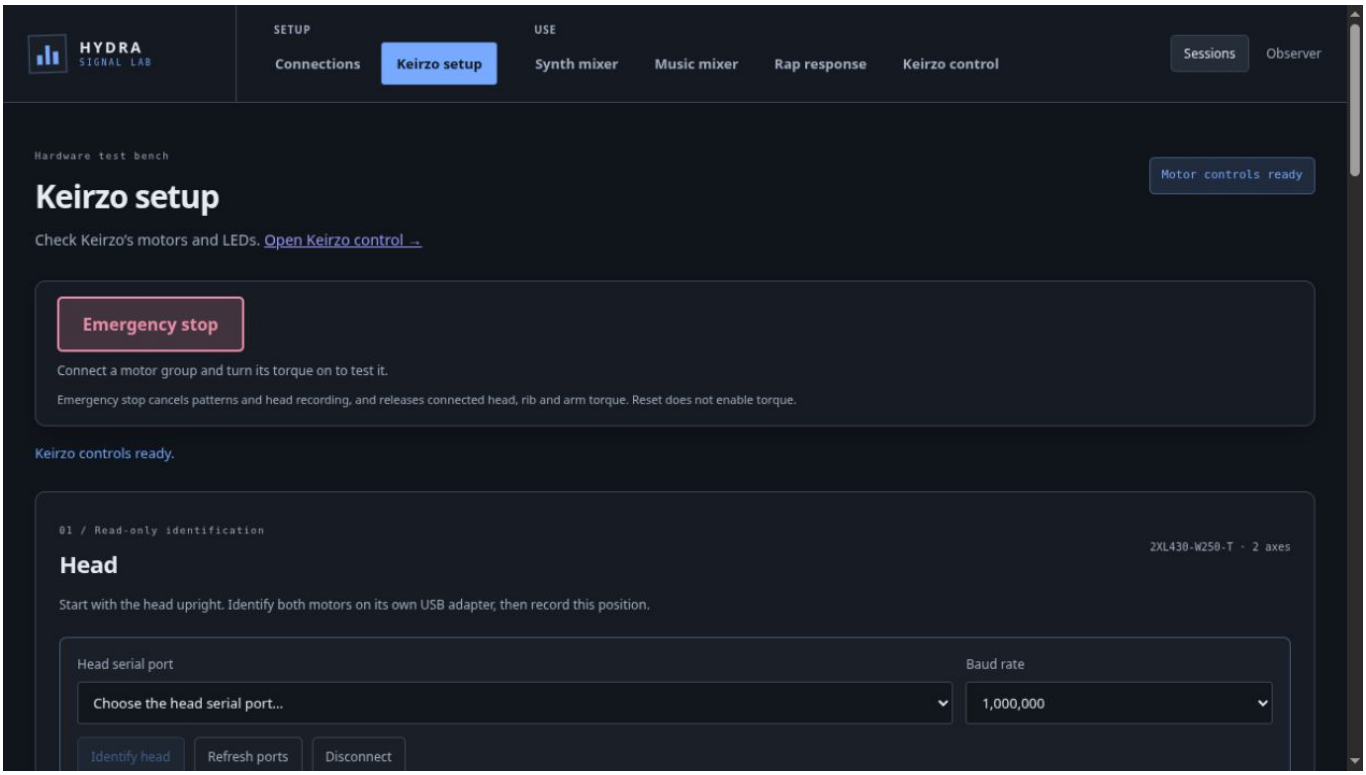


Figure 10. Keirzo setup with the robot disconnected. Connection status and missing devices are expected in this screenshot; the controls operate real hardware when it is connected.

Know which adapter belongs to each part

The head, ribs and arms have three separate USB serial connections. Their identities are saved in config/robot-devices.json using USB vendor ID, product ID and serial number. The current Linux path, such as /dev/ttyUSB0 or /dev/ttyACM0, can change after reconnecting. The saved USB identity lets the dashboard find the same adapter after moving it to another socket or computer.

Group	Configured adapter identity	Motor IDs and baud
Head	ROBOTIS OpenRB-150; serial 6430 A5355055344E312E3120FF0C0B25	Discovered two-axis head; recorded IDs 0 and 1; 1,000,000 baud
Ribs	FTDI adapter; serial FT78LBUJ	20, 21, 22, 25, 26, 27; 1,000,000 baud
Drumming arms	FTDI adapter; serial FT7WBEBD	0, 1, 2, 3; 1,000,000 baud
LEDs	Pimoroni Plasma 2040 over USB MIDI	Browser MIDI; independent of motor serial connections

The head and drums reuse motor IDs 0 and 1 on different buses. Identify the adapter, not just the motor number. Discovery only reads the computer’s list of serial ports; it does not open a bus or test its motors. A missing or ambiguous USB identity is not replaced with another adapter. Manual selection is available, but a selected port still needs a successful connection check.

1. Connect and power the intended hardware. Close Max, interaction.py and other programs that could own its serial adapters. The legacy interaction.py bridge opens the rib and drum ports and is not needed by these direct dashboard controls.
2. Open Keirzo setup and use Refresh ports after plugging in or reconnecting an adapter. Confirm the selected adapter for each group against its USB identity.
3. Use Identify head for the head and Check connection for ribs and arms. These checks use ping/register reads and do not enable torque or move a motor. Inspect every motor reply, position, torque state and reported error.
4. Resolve missing motors or a mismatched head calibration before using torque or test controls. The tables show timestamped measurements, not continuous position tracking; check again when the physical state may have changed.

Inspect devices from a terminal

```
.venv/bin/python tools/robot_devices.py
.venv/bin/python -m serial.tools.list_ports -v
```

These listings help distinguish an absent USB adapter from a motor that does not answer. On Ubuntu, an actual serial permission error may require dialout membership and a logout/login after a group change. Do not run the dashboard as root. If the adapter appears but no motors answer, verify the selected role, power, baud rate and that another serial client is closed. Changing a group’s port requires another check. The dashboard prevents two groups from opening the same adapter.

The Linux port has been tested with virtual MIDI, synthetic audio and simulated serial hardware. The robot and Hydrasynth were not connected for this Linux installation. Earlier physical observations in the repository refer to the recorded rig on macOS; they do not establish that a new laptop, changed mounting, or current sequence has passed a physical acceptance run.

CHAPTER 12

Stopping torque and disconnecting

A stop button, torque release and serial disconnect do different things. Torque is the motor’s holding force. A motor can stop receiving new movement commands while remaining powered and holding its position. Conversely, releasing torque can allow an assembly to fall or move under its own weight. Support the head before deliberately releasing its torque.

Action	What it does	What it does not do
Top Stop in Keirzo control	Cancels startup or stops robot effects and automatic drumming. Normal LED performance turns off. Stops future rib/head commands; healthy moving drum arms may finish their return.	Does not turn off motor torque or interrupt an already-issued rib/head move. Mixer audio has its own Stop control and is not stopped by this robot control.
Stop in Automatic drumming	Discards upcoming drum hits and lets healthy arms already moving complete their return.	Does not stop LED, rib or head performance. Does not release arm torque. Mixer audio has its own Stop control and is not stopped by this robot control.
Release ribs / Release arms / Release head	Requests torque off for that group. Head release stops head scheduling; arm release cancels pending drum motion/return.	Does not release unrelated motor groups.
Disconnect for a motor group	Closes its serial connection; arm disconnect also cancels the pending return.	Does not turn torque off.
Emergency stop	Cancels live robot patterns, queued motor commands and drum returns, stops head range recording, and requests torque off on already-open, verified buses.	Does not open unknown adapters or guarantee a torque-off write reaches an unavailable motor. Mixer audio has its own Stop control and is not stopped by this robot control.
Reset emergency stop	Clears the software latch.	Does not enable torque, replay cancelled commands or restart drumming.

Use the ordinary stop for planned pauses

Press the top Stop in Keirzo control when ending both robot performances. A head or rib movement already sent to the motor finishes its profile; holding torque stays on. Stop is therefore a controlled sequencing pause rather than an instantaneous mechanical stop. Press Start robot again when ready to prepare the selected effects. Start drumming is separate and must be used when drum hits are wanted.

Changing dashboard workspaces is not a stop: music, connections and robot performance remain active while you navigate. The live browser sequences need the page to stay open. Closing or reloading the page ends browser sequencing, but it is not a reliable substitute for deliberate stopping and releasing the intended groups. Server-run manual strike/return tests can continue across tab changes. A loss of the automatic-drumming heartbeat ends further queued performance after its bounded timeout.

Understand the emergency stop latch

Emergency stop appears in both Keirzo setup and Keirzo control. It is a software stop sent over serial, not a physical power cutoff. The server attempts torque release on connected, verified rib, arm and head buses and reports failed writes. If a motor cannot be reached, the software cannot release it through that connection. Torque off also removes support, so consider the actual head and arm position when operating the rig.

The latch persists across dashboard tabs and page reloads while the server remains running. Correct the cause, read any reported torque-off failures, then use Reset emergency stop. Explicitly prepare/start again after resetting; no cancelled performance resumes by itself. A server restart starts with motor output available but does not open ports or issue motor commands. Restarting the process is not a substitute for a controlled recovery procedure.

Finish a robot session deliberately

1. Use the top Stop and confirm the live performance is stopped. Allow healthy in-progress returns or profile moves to complete when this is an ordinary planned shutdown.
2. Support the head before using Release head if it should no longer hold its pose. Use Release arms and Release ribs for the groups whose torque should be removed. Read the result of each action; a button click is not proof that every motor received it.
3. Disconnect the relevant ports once their intended torque state is established. Stop the dashboard with Ctrl+C in its terminal when finished.
4. Investigate any failed release or lost connection at the actual robot. A serial disconnect or closed browser does not establish the physical state of the rig.

CHAPTER 13

Manual rib drum and LED tests

Rib cage

Check connection must read all six rib motors before grouped movement or torque-on becomes available. Press Rib torque on when ready to hold the ribs, set Rib cage position and Duration (ms), then press Test rib cage. Editing a number or slider prepares the requested test; it does not itself send the movement. Expand Individual rib motors to test a single motor with Test position. The Command log records requested commands and results.

The grouped position uses the original Max scale from 0 to 127 and mirrors the two sides. At 0, motors 20-22 target 2450 and motors 25-27 target 1700. At 127, those targets become 4090 and 0 respectively. The default grouped duration is 2500 ms. The test interface accepts 100-5000 ms; this is an interface policy, not a statement of the motor’s mechanical limits.

Before a direct move, the service rechecks the motors and requires torque on, position operating mode and time-based profiles. It does not change those modes for you. It writes the profile duration and goal position and checks the replies. A successful register acknowledgement confirms receipt of the command, not physical arrival. Release ribs affects only these six motors and remains available while emergency stop is latched. On a verified rib bus it attempts each release even when a motor fails to reply.

Drumming arms

1. Select the separate drum adapter and press Check connection. Confirm all four motor replies, torque states and errors.
2. Press Arm torque on + rest. This enables holding torque and returns all four arms to configured rest over two seconds, including arms that were already powered or outside their normal playing ranges. Wait for the confirmed ready/rest result.
3. Press Test strike for one part. The test first raises that arm to rest over 500 ms, checks that it is within 20 ticks of rest, sends the configured hit, and returns it to the same raised rest with a 150 ms profile.
4. Wait for the complete result before another test. Only one manual strike runs at a time; clicking again does not queue another hit. If an arm is outside its playing range, use Arm torque on + rest before striking.

Part / motor ID	Allowed positions	Configured rest / hit	Hit profile
Kick / 0	2100-2450	2200 / 2370	150 ms
Ride / 1	1850-2125	1850 / 2000	150 ms
Snare / 2	2070-2350	2300 / 2150	150 ms

Part / motor ID	Allowed positions	Configured rest / hit	Hit profile
Hi-hat / 3	2410-2600	2600 / 2430	200 ms

These are raw Dynamixel ticks from the repository’s Max bounds and stroke profiles, not degrees. Ride and Snare were physically exchanged: motor 1 is Ride, motor 2 is Snare, while their motor-specific positions remain attached to those IDs. The hi-hat uses the in-range second source profile rather than expanding its bound to include an old out-of-range profile. Advanced manual position controls send a single move without an automatic return; choose them deliberately.

Release arms affects IDs 0-3 on the selected drum bus, removes holding torque and cancels a pending return. Disconnect also cancels the return but leaves torque unchanged. The controller checks rest arrival rather than treating elapsed profile time as proof of position. The automatic performance engine additionally insets boundary rest goals for settling tolerance, including hi-hat rest at 2590.

LED controller

Connect the Pimoroni Plasma 2040 over USB and allow MIDI access in the browser. Keirzo setup finds a Plasma 2040/Pimoroni output automatically; it does not fall back to the synth. It normally appears as Plasma 2040 CircuitPython usb_midi.ports[0]. A new browser needs its own MIDI permission. Use Retry MIDI connection after granting permission if access was denied.

Choose a non-black Colour and Brightness above 0%, then press Test colour. Press LEDs off to send black. Connecting MIDI or changing the colour picker alone does not send a test. If discovery succeeds but the strip stays dark, check the hint beside the controller and the board’s LED program: a MIDI device in the list proves USB discovery, not that its LED firmware is running. The current automatic matching assumes one matching LED board.

The checked-in lights/code.py fills 96 LEDs using a two-message RGB command on MIDI channel 1: Note On with note 1 and red as velocity, followed by Polyphonic Key Pressure with green as note and blue as pressure. Components are 0-127 and scaled by the firmware. LED control is independent of motor torque and the serial connections. Reconnecting the board does not automatically change its colour.

CHAPTER 14

Head calibration and preparation

Powered head movement requires a saved range for the same physical head assembly and a matching connection. Copy var/robot/head-calibration.json from the computer previously used with that same mounted robot, or record the intended travel manually. The file lives under Git-ignored var/, so cloning the repository does not bring it to a new laptop. A saved browser session is not a replacement for this file.

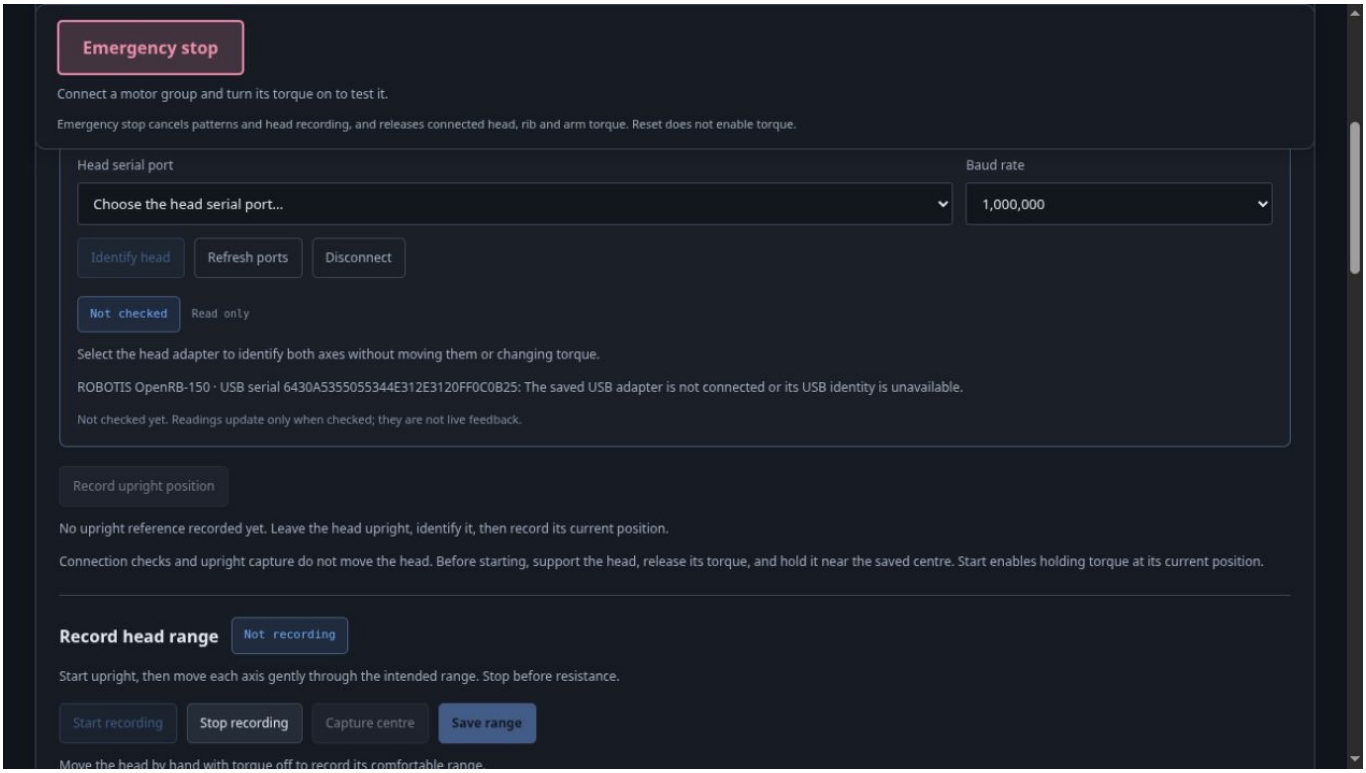


Figure 11. The Head identification and Record head range controls. With hardware disconnected, the manual shows their location rather than fabricated motor readings.

Identify before recording

The documented rig uses a two-axis 2XL430-W250-T, model 1090, on its own OpenRB-150 bus at 1,000,000 baud. Recorded IDs are 0 and 1, but discovery reads the responding IDs and model rather than assuming factory IDs. The model’s factory baud is 57,600, which matters only for a different unconfigured setup. Select the actual adapter and configured baud, then press Identify head. Review the two measured positions, torque, modes and errors.

With the head upright, Record upright position saves a read-only reference in server memory. It does not centre the head, enable torque or establish travel limits. The reference is lost on server restart or a change of adapter, baud or identified axes. Motor factory limits of 0-4095 are not safe mounted-head limits. Do not use historical positions as a generic calibration for another assembly.

Record travel with torque off

1. Support the head. If torque is enabled, use Release head and confirm torque off before moving it by hand. Both axes must respond with torque off before recording starts.
2. Place the head upright and press Start recording. The centre is taken from an upright reference for this connection when available; otherwise it uses the starting positions.
3. Move each axis gently through the travel intended for use, stopping before resistance. Watch the Current, Minimum and Maximum readings and name the axes as you identify them.
4. Press Stop recording. If needed, return the head to the desired centre and press Capture centre to read both motors again.
5. Review both axes, then press Save range. Each needs a nonzero span and a centre inside that span. The previous saved range is replaced only after validation and a successful file write.
6. Keep a backup of the calibration with this same mounted head. On another laptop, confirm that the loaded range reports a matching connection after identification.

Recording runs on the server and continues across workspace changes. A read error, torque becoming enabled, connection change or emergency stop stops it; captured samples remain available, but unsaved recording state disappears when the server restarts. Saving a range neither enables torque nor starts performance. Recorded positions describe observed individual travel, not proof that every simultaneous two-axis pose is mechanically safe.

Prepare the head for powered movement

A head with torque off can rest below its saved range. Support it, release torque if necessary, and gently hold both axes near their saved centre and inside their saved ranges. Identify head reads the current pose without moving it. In Keirzo control, enable the Head, choose Manual Busy-ness 0 for an initial modest activity setting, and press Start robot. Preparation checks the saved identity and current pose, preloads that measured pose and enables holding torque. It does not command a centring move. Keep supporting the head until holding torque is confirmed, clear of the moving joints; the selected head performance follows preparation.

Powered commands require the saved adapter identity, baud, two motor IDs and model to match. They also require healthy readbacks, position mode 3, time profiles and an in-range measured pose. They are blocked during range recording, for secondary-ID aliases, or when automatic torque-on by goal update is enabled. The application does not change EEPROM settings or factory position limits to bypass these checks. Older calibrations tied only to a port path may not follow an adapter to a new port.

For historical reference only, the recorded assembly had centres 2075 and 2529, with ranges 1408-2654 and 1948-3071 for IDs 0 and 1. Always use the current verified range displayed in Setup. The documented rig has also shown small-movement vibration whose physical cause remains unresolved. The current profiles avoid very slow motion; position gains were not changed. Its wider flowing sequence still needs a physical check on the actual assembly.

For diagnosis, GET /api/robot/head/diagnostics samples an already-open identified head without connecting it or moving it. It reports measured position, internal trajectory, goal, output, inferred load and supply voltage. Ordinary status tables use cached readings. Keep the exact diagnostic and visible physical behavior together when reporting a problem; a requested target alone does not show that the head arrived.

CHAPTER 15

Keirzo control coordinated lights ribs and head

Open Use > Keirzo control or http://127.0.0.1:8790/#robot-control. Choose Follow > Music mixer or Synth mixer. The BPM and beat indicators show the chosen player's clock; there is no independent robot tempo. The link next to Follow opens that mixer. The selection also applies to Automatic drumming, so changing it changes the source for lights, ribs, beat-synced head motion and drums together.

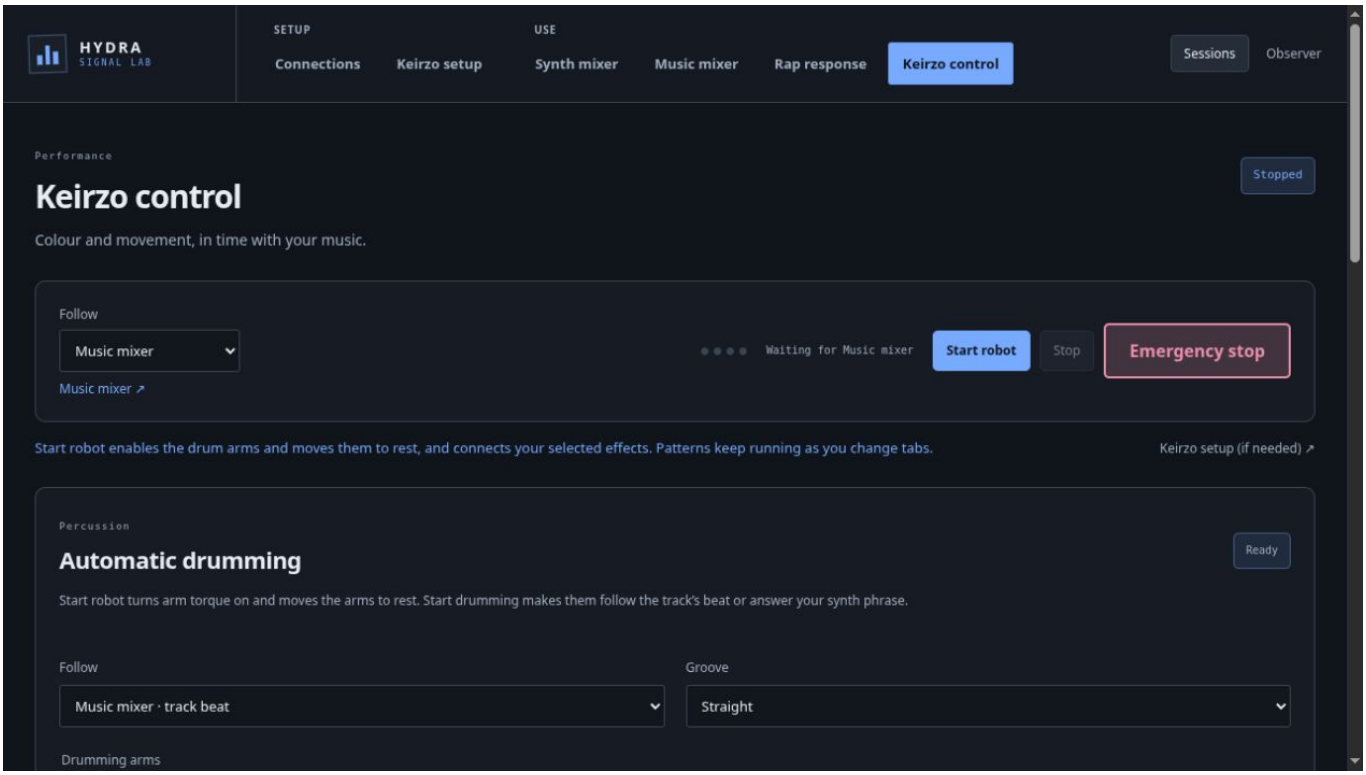


Figure 12. Keirzo control groups the performance transport, source following, LED palette, rib breathing and head activity. Start robot prepares real motor hardware.

Prepare and start

1. Choose the Follow source and configure the effects you intend to use. Prepare a saved matching head range and support the head near its centre if Head Enabled is selected.
2. Press Start robot. Read the preparation status while it finds connections and enables required torque. It always prepares all four drumming arms at raised rest, even if LEDs are the only selected effect or all effects are disabled.
3. Wait for preparation to finish before expecting performance. Missing arms, an active strike/drumming sequence, motor mode errors or a mismatched head range can prevent startup. Stop drumming before trying rest preparation again.

4 Start playback in the chosen mixer for synced effects. Press Start drumming separately if actual drum hits are desired.

5 Use the top Stop to stop both robot effects and automatic drumming. Navigate between workspaces freely during a performance; the dashboard page must remain open.

The synchronisation comes from the dashboard's playback clock, not microphone beat detection. When playback stops, synced effects wait. Changing Follow cancels queued drum hits; explicitly press Start drumming again for the new source. A saved session restores preferences passively and does not start robot playback.

LED performance

Enable the LEDs, choose Aurora, Firelight, Ocean or Candy, or edit the four palette colours. Solid colour uses the first colour. Palette cycle advances through the palette, and Beat pulse varies brightness on the beat. Change colour every selects ¼, ½, 1, 2, 4, 8 or 16 beats. Brightness scales the overall output. Start robot finds the same automatically detected Pimoroni controller used in Setup. It also prepares the arms even for an LED-only performance.

Rib breathing

Enable Rib breathing, set Minimum position and Maximum position on the 0-127 scale, and choose how many beats pass before moving to the next position. Four beats means four beats outward and four back: an eight-beat full cycle. The motion uses the same mirrored mapping and bounds as Setup. Start robot checks all six rib motors and turns rib torque on if needed. Use Setup to change an adapter or investigate a reported connection/mode problem.

Head motion

Head Enabled includes the calibrated head in startup. New preferences leave it disabled. Manual Busy-ness ranges from 0 to 100; 0 still produces calm ongoing movement and is not an off setting. Higher values increase span and activity. Turn Enabled off to stop scheduling after current movements finish. Auto varies activity gently between 15 and 65 independently of audio energy or volume.

Beat sync starts enabled. The first gesture waits for a fresh beat and profile lengths follow whole beats when that keeps a moderate pace; later movements continue when each axis finishes. Turn Beat sync off for ongoing movement at its own pace even with the mixer stopped. The two axes use overlapping sweeps and turns with staggered turnaround times rather than fixed pauses. Missed cues are skipped, not replayed in a burst.

The chosen activity span retains margins within the saved range: targets reach up to 22.5% of each recorded half-range at Busy-ness 0 and 72% at 100. Individual moves are limited to 675 ticks, profiles to ten seconds, and the server lengthens profiles for larger moves with its conservative peak-speed bound. An active goal is not replaced midway through its profile. These software bounds do not establish collision-free mounted motion; verify the real assembly before relying on continuous performance.

Stop leaves holding torque on and allows an issued move to finish. Release head is in Keirzo setup and removes that support. Keep this distinction in mind when turning off head activity at the end of a session.

CHAPTER 16

Automatic drumming pattern timing and recovery

Automatic drumming is the percussion panel within Keirzo control. It has its own Start drumming and Stop buttons. Start robot prepares the arms, lights and selected movement; it does not start hits. The panel can also prepare the required arm connection when Start drumming is pressed. All four arms must pass preparation, even when you intend to play only selected parts.

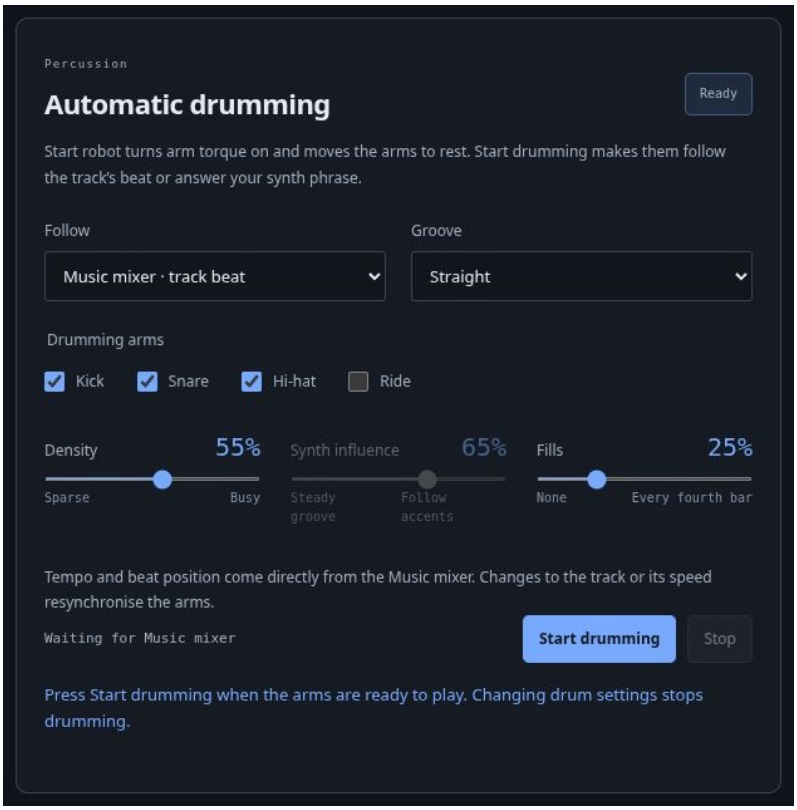


Figure 13. Automatic drumming: select a source, groove and arms, then adjust Density, Synth influence and Fills before starting. Timing limits can remove hits.

1. Choose Music mixer · track beat or Synth mixer · rhythm and notes. This Follow selection stays in sync with the main Keirzo control selector.
2. Enable the desired kick, ride, snare and hi-hat arms. Choose Straight, Broken beat or Half-time.
3. Adjust Density, Synth influence where applicable, and Fills. Fills introduce variation every fourth bar.
4. Press Start drumming. If the selected source is stopped, the panel waits. Start that source’s playback and watch preparation/ready status before expecting hits.
5. Use the drum panel Stop for drums only, or the top Stop to end all robot performances. Editing a drum setting stops drumming; review the settings and press Start drumming again when ready.

What the drum source actually follows

Music mixer supplies the playing track’s BPM and beat position. Its grid is estimated from requested generation BPM and track start; it is not a detected acoustic beat grid. Synth mixer uses actual note events, including probability, rests, chords and repeats, and the preceding bar’s accents shape the next drum bar. A chord counts as one onset. Synth influence moves from a steadier groove toward those accents. Neither source analyses microphone sound.

Each individual drum is limited to one hit per second, including high-density settings and fills. Different drums may strike together; there is no global spacing interval between different arms. The browser thins the requested pattern and the server independently enforces the same per-arm limit. Expired, duplicated and physically infeasible hits are dropped instead of accumulated.

Rest confirmation and fault handling

When required, startup enables arm torque and moves all four arms to rest over two seconds. Selected arms then receive the normal 500 ms preparation. Each must be inside its strike range and within 20 ticks of rest before a hit is allowed. Nominal profile time is not taken as proof of arrival: a healthy arm gets up to one additional second to settle, with position and health checks every 50 ms or when the bus is available.

A healthy powered arm found outside its playing range has queued hits removed and returns to rest over two seconds. Boundary rest targets are inset by 10 ticks for tolerance, including hi-hat at 2590. It rejoins future hits only after an in-range rest readback. A settling timeout, lost torque or motor fault pauses that arm and retains the reason; other healthy arms can continue. Resolve the reported cause and start a new session for the paused arm. Uncertain movement writes are not automatically retried.

Source stops, track changes and timing discontinuities cancel queued hits. While Start remains active, valid playback can prepare and rejoin automatically. Fast playback-rate changes suspend drumming until the clock settles; synth tempo changes may also need resynchronisation. An adapter disconnect, explicit torque release or emergency stop ends the whole drumming session and requires a new explicit Start. Resetting emergency stop never resumes it.

Timing and diagnostic limits

The browser refreshes a short server queue with a one-second heartbeat timeout. Hits are submitted 650-950 ms before their intended contact. Each arm uses an absolute target contact time: kick, ride and snare start their 150 ms strike profiles before it, and hi-hat starts 200 ms before it. Return profiles are 150 ms. The controller checks/stages the arm before timed dispatch; late or cancelled goals are dropped. A normal stop lets healthy in-flight strokes return, while release, disconnect and emergency stop cancel further motion.

The original Max patch aligned contact by delaying 400 minus the stored hit time, then striking for that time. This implementation schedules directly from the desired contact time and does not add another 400 ms. These are intended profile timings, not measurements of acoustic contact. Dispatched-strike and skipped-hit counts aid diagnosis, but physical contact alignment and continuous multi-arm operation remain to be checked on Keirzo. Keep the selected groove, tempo, affected arm, status reason and observed motion when reporting a failure.

CHAPTER 17

Native Hydrasynth MIDI Linux audio and older routes

Choose the MIDI path intentionally

The browser Synth mixer can send through Web MIDI independently of the native controller. make dashboard-sonify is enough for browser sound and, with browser MIDI permission, browser-driven Hydrasynth output. make dashboard starts the native hydractl service for hardware telemetry and the local command broker. The native service and browser output have separate connection behavior; selecting one does not turn USB MIDI into an audio interface.

1. Connect and power the synth, start with monitor volume low, and connect its analogue output to headphones, speakers or an audio interface. Hydrasynth USB carries MIDI commands, not the synth’s audio.
2. On the synth, match MIDI RX to the intended channel. Browser output uses channel 1; native commands default to 1 but accept --channel 1-16. For the documented acceptance rig, MPE is off.
3. Use PARAM RX=NRPN for dashboard catalog mappings. Named native timbre commands use CC and need PARAM RX=CC. Choose PARAM TX=CC or NRPN according to the feedback you want to observe. Enable PGM CHG RX for remote patch changes and PGM CHG TX for front-panel patch tracking.
4. List native endpoints, then launch the live dashboard if you need its telemetry/broker. Stop an existing dashboard before launching another on port 8790. Inspect the selected destination and source before sending a note.

```
./build/hydractl list
aconnect -l
make dashboard
```

A native destination can be selected with --device using a unique name or Linux ALSA client:port address. Numeric addresses can change after reconnecting or rebooting. Prefer a unique name for a saved launch command; ambiguous matches fail. To pass native options through Make, for example, use make dashboard DASHBOARD_ARGS='--device "unique device name" --channel 1'. Replace the example with the listed name.

The native service detects MIDI loss, stops scheduling and exits. Reconnect the intended synth and restart the live dashboard/service; there is no native automatic reconnection. The browser Web MIDI path can rediscover its saved device, but requires Play again after disconnection. Physical USB removal can prevent delivery of note-off messages, so inspect the instrument for held notes when recovering.

Native command reference

Run these sending examples only after selecting the intended synth and establishing a listening level. Patch selection discards unsaved edits in the current synth buffer. The following are independent terminal commands; while the live dashboard owns a service, use its command client for shared scheduled operations rather than accidentally starting a second service.

Command	Purpose
./build/hydractl note C4 --velocity 72 --duration 0.5	One bounded note; note names or MIDI numbers are accepted.
./build/hydractl sequence C4 E4 G4 C5 --step 0.2 --gate 0.7	A short note sequence; step is seconds and gate is 0-1.
./build/hydractl timbre list	List the named normalized CC controls.
./build/hydractl timbre cutoff 0.70	Set a named value from 0-1 with PARAM RX=CC.
./build/hydractl cc 74 96	Send a raw CC number/value from 0-127.
./build/hydractl nrpn 1234 8192	Raw NRPN example; address/value are 0-16383. Select a documented parameter before use.
./build/hydractl patch A 1	Select bank A, patch 1; patch numbers are 1-128.
./build/hydractl bend -0.5	Normalized pitch bend from -1 to +1; return to 0 afterwards.

```
.venv/bin/python -B tools/hydra_command.py --pretty '{
  "v":1, "id":"state-1", "op":"state"
}'

.venv/bin/python -B tools/hydra_command.py '{
  "v":1, "id":"short-note", "op":"play",
  "notes":[60], "velocity":72, "duration_ms":500
}'
```

The client talks to the live dashboard’s private local Unix socket and reads its generated token automatically. Demo mode has no native command broker. Each request carries an ID so the response can be matched. Scheduled play supplies its own note-offs; direct note_on clients must provide note_off. The service also supports overlapping scheduled notes and linear CC ramps, state/status queries, job cancellation and user takeover protection. A physical control movement normally blocks competing non-forced writes to that parameter for three seconds. The technical reference in docs/reference.md documents the full NDJSON protocol.

Current native cleanup caveat: hydractl panic sends CC64, CC123 and CC120. The persistent service’s panic, shutdown and failed-note-off cleanup also send CC120, with service-wide panic covering all 16 channels. The repository’s Hydrasynth mapping assigns CC120 to ARP Octave, so do not treat native panic as a transparent routine reset that preserves the patch’s arpeggiator settings. Browser All notes off uses sustain-off and All Notes Off without CC120. This existing native/documentation discrepancy needs hardware validation; retain the synth state when investigating it. MIDI cleanup is separate from Keirzo Emergency stop.

Audio capture and level checks on Linux

```
./build/hydra-audio list --json
./build/hydra-audio meter --device 'Revolution 2x2' --channel 1 --duration 2 --json
# Or replace this UID with the actual UID in your list:
./build/hydra-audio meter --uid 'hw:CARD=USB,DEV=0' --channel 1 --duration 2 --json
```

hydra-audio is an input meter. It neither records an audio file nor supplies native synthesis/playback; the dashboard plays through the browser. The default selection is a device containing Revolution 2x2, channel 1, for ten seconds. Explicit durations are 0.1-3600 seconds. Channels are one-based. --device selects by name; --uid selects the listed ALSA PCM identifier and can disambiguate a device. A card-name UID is more stable than a numeric card index.

Play a known signal into the selected interface input and inspect frames, elapsed time, RMS, peak, clipping and non-finite counts. Clipping means a sample at or above an absolute value of 1.0, or 0 dBFS; use the interface/synth gain controls to resolve an excessive input level. Healthy physical acceptance requires plausible nonzero capture duration and counts, finite values, a signal above idle noise and no clipping at the intended level. A synthetic passing test cannot verify the physical cable, input channel or gain.

Direct hw: capture can be busy when another program owns the interface. Close the conflicting capture application or select --uid default or an explicitly configured PipeWire/PulseAudio ALSA PCM. No change of system audio server is required. Desktop/custom route listing intentionally reports zero channels/rate when capability is unknown, without opening that backend. Selection and metering determine the usable format. arecord -l lists capture hardware; /dev/snd/seq access is needed for native MIDI, including in containers.

Earlier interfaces and legacy integration

The Previous version link in Synth mixer opens #synth-legacy, retaining its older per-recording mapping tabs and presets. The older movement-driven electronic drum interface remains at #drums and uses a six-voice browser kit; it is distinct from Keirzo’s four physical arms. Observer is at /observer/ and can display native/demo telemetry or the Signal Lab feed. These views share the dashboard, but their output and source choices still matter.

The direct Keirzo motor controls do not require the sibling drumming-robot repository, Max or interaction.py at runtime. Their ranges and strike timing were derived from those older files; do not launch the old serial bridge alongside the dashboard merely because the manual mentions that provenance. After connecting real hardware on Linux, complete docs/hardware_acceptance.md for MIDI feedback, notes, scheduling, disconnects and audio, and perform the robot checks in docs/robot_control.md. Record the machine, build, device firmware, exact command/settings and physical result.

CHAPTER 18

Daily operation updates and saved material

A consistent start and finish

1. Start the server from the correct checkout using the same study-directory override, if any, that you used previously. Read the startup lines showing its study and backup locations.
2. Open the usual local browser address. Check the desired input and output in Connections before playback. Reconnecting a device and selecting a saved session do not by themselves resume playback.
3. Before participant work, open the researcher view and confirm storage health, the intended laptop ID and participant session. Keep the machine on reliable power and check available storage.
4. At the end, stop audio and robot activities, let pending study records reach Saved locally, and then stop the foreground server with Ctrl+C. Collect the completed backups as described below.

Update an existing laptop

```
git status --short
git pull --ff-only && bash tools/setup.sh
make dashboard-sonify
```

Stop the running dashboard before updating. Review git status first: local source edits need to be preserved or reconciled, not discarded. The fast-forward pull refuses an unexpected history merge rather than guessing how to combine it. Rerun setup after pulling to update Python dependencies within the supported versions and rebuild native tools. Whisper is installed automatically if missing. It preserves the existing usable environment, model, key and local data. A failure during an update is a reason to resolve the reported problem before beginning a participant session.

All six laptops should use the same application revision during a study. A clone or pull receives only committed, pushed files. It does not carry an uncommitted local fix, .env, .venv, generated media or the normal study folder. Record the revision with git rev-parse HEAD when checking a prepared machine.

Know what is saved where

Material	Location and transfer rule
Application and bundled assets	Git checkout. Update through Git. The bundled recordings are compressed and read directly.
API credential	.env in this checkout. Keep private; provision separately on each laptop.
Speech model	var/whisper/small. May be copied to save downloads.
Generated music and rap assets	Project var folders, including var/elevenlabs and var/rap-response. Preserve these if you want saved job references to remain playable.
Mixer settings and Sessions library	This browser’s local storage. Export selected/all sessions to portable JSON before changing browser, profile or origin.
Study journal and microphone media	The configured study-data directory. Independent of the Sessions settings library.
Head calibration	var/robot/head-calibration.json. Local and specific to the physical robot.

Saved Sessions JSON captures settings and job references; it does not embed all referenced audio. Moving only the JSON to another laptop cannot recreate missing generated tracks. Study export bundles serve an analysis/archive purpose and are not an automatic import mechanism for the application’s music library. Keep a separate operational copy of generated assets when migrating a working installation.

Use the same browser address consistently: http://127.0.0.1:8790 and http://localhost:8790 have different browser origins, as does a different port. Their local settings and permissions can therefore differ even though they reach the same server. Clearing site data, using another browser profile or opening a private window can make saved settings appear to have disappeared.

CHAPTER 19

Study recording and participant handovers

Study recording and local backup creation are enabled automatically. Participants do not have to press Record for ordinary interaction logging or perform a save/export step. Explicit microphone recording remains an intentional user action. Researcher controls are shown at http://127.0.0.1:8790/?researcher=1. This query option changes the presentation and is not an authentication mechanism.

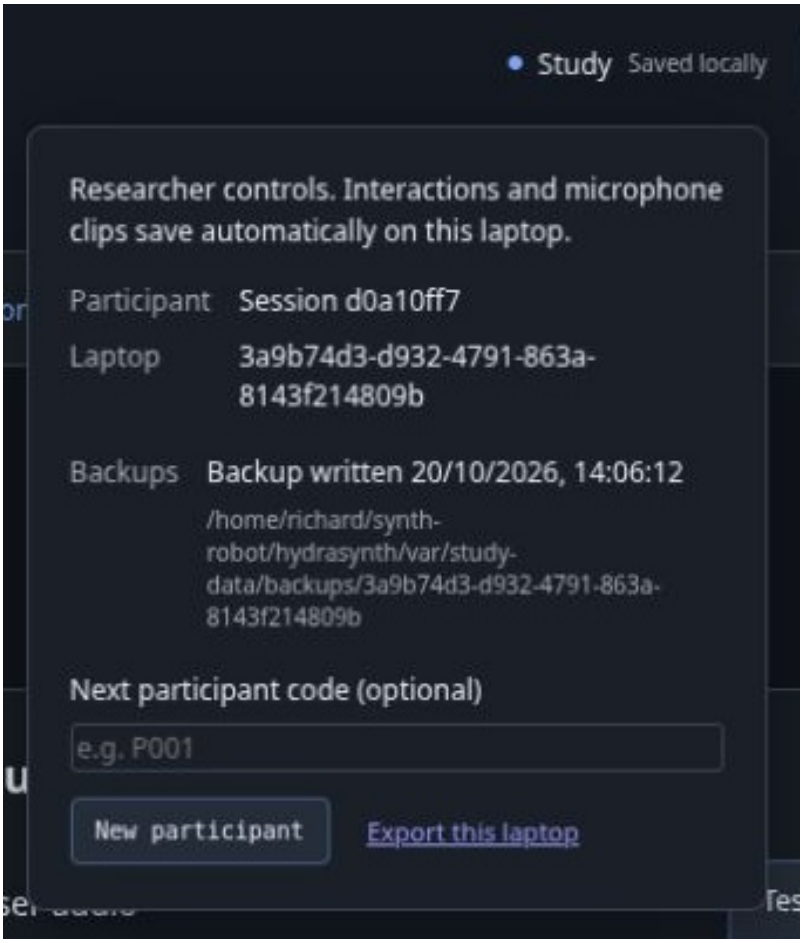


Figure 14. The researcher Study panel exposes recording health, pending work, laptop identity, participant handover and export controls. Normal participant use keeps this panel hidden.

Create a handover in the participant’s own tab

1. In the same browser tab the participant will use, open the address with ?researcher=1 and expand Study.
2. Enter an optional participant code and choose New participant. Confirm the displayed session before continuing.
3. Remove the researcher query option to return to the ordinary participant view. Keep using this tab.

4 At the next handover, repeat the procedure in that same tab. A second researcher tab has its own session and cannot mark a handover in the participant tab.

Without an explicit handover, the recorder sees a continuous session and cannot infer that somebody else has started. A page reload preserves the tab’s session while creating a new client run. Activity without a participant code is still recorded. These participant sessions are different from the header’s Sessions library, which stores mixer/settings snapshots.

What the records describe

The recorder captures interface actions, accepted mapping configurations, playback and output events, browser/backend errors, numeric live input, generation settings and job transitions. Microphone clips include original recordings and relevant transcription submissions; transcripts, supplied words, generated lyrics and audio references are retained. Mapping configuration and playback revision IDs help relate a setup to the period when it was used. Canceled microphone attempts are also retained when study recording is enabled; canceling a rap is not deletion of the study capture.

Events include device, participant-session and client identifiers, timestamps, elapsed time, sequence information and application revision. Some backend hardware events have no known participant and remain unassigned; they can be aligned by laptop and time. A command or scheduled output record does not prove that a speaker sounded or a robot physically made contact. Recording does not add continuous ambient microphone, screen or camera capture.

Storage and health

On Linux the default is $XDG_DATA_HOME/keirzo/study-data, or ~/.local/share/keirzo/study-data when XDG_DATA_HOME is unset. macOS uses ~/Library/Application Support/Keirzo/study-data. KEIRZO_STUDY_DIR or --study-dir chooses another local directory. The development machine may have been started with var/study-data explicitly; use the actual startup message and Study backup path rather than assuming a location.

Each fresh study database creates its own installation ID. Do not copy the database between operational laptops. Keep live databases on local storage rather than inside a cloud-synchronised folder. Browser IndexedDB queues events and completed clips until the backend acknowledges them; the backend persists the journal in SQLite with separate audio files. This works without internet access, but it still needs the local server and writable disk.

Watch for pending recordings and storage errors in Study. Keep the tab open while work is pending, and resolve a full disk or denied folder access before closing. An abrupt browser/process exit can lose an unfinished microphone recording or events that have not reached persistent storage. The system does not automatically delete old sessions or recordings, so plan disk space and retention for the study.

For development, --no-study-recording disables recording entirely. --no-study-backups disables only the automatic backup bundles; it leaves primary recording enabled. Do not add either flag to participant launch commands by habit. Keep laptop clocks synchronised before a session; timestamp alignment cannot repair unknown clock drift after the fact.

CHAPTER 20

Collecting exporting and combining study data

Automatic backups are incremental

Automatic backups run at server startup, every two minutes and during normal shutdown. Completed ZIP files appear under study-data/backups/<laptop-id>/. Each adds new events, microphone clips and new or changed generated audio. Retain every ZIP in that folder: the newest file is not a complete substitute for the earlier files. Failed backups are retried while primary local recording continues.

These automatic copies remain on the same laptop until you copy them to another device. They do not protect against loss of the entire laptop on their own. There is no automatic upload. Data still pending in the browser becomes eligible for backup only after the backend has saved it. These ZIPs can contain participant transcripts and microphone audio. The application does not encrypt the archives; handle collection drives as participant data.

Collect onto USB after a session

1. Open the researcher Study panel in the participant tab. Stop the activity and wait until pending recordings have drained and the status reports Saved locally.
2. Check Backups for a recent successful write and no error. Saved locally confirms primary storage, not ZIP backup success. If backups fail, use Export this laptop while the server is still running and verify that archive. A clean shutdown alone does not prove the final backup succeeded.
3. Stop the server normally with Ctrl+C and allow the final backup to finish. Do not force-kill the process just to speed up collection.
4. Locate the backups folder shown by the server/Study panel and copy the entire backups folder to your collection drive. Preserve every per-laptop subfolder and completed ZIP.
5. Repeat for each laptop. Check that the expected six installation folders and files are present on the collection drive before removing it.
6. Keep the originals until the collected files have been verified and the study’s retention process says they can be removed.

A complete manual snapshot

With the server running and the browser queue drained, choose Study > Export this laptop. This downloads a separate complete snapshot of all recorded sessions on this installation, leaving the originals intact. It does not reset automatic backup progress. The archive contains exact JSONL, analysis-friendly CSV, participant-session records, linked audio, job metadata and a manifest with file hashes and unavailable-asset details.

A job that is still generating may not yet have completed audio. The manifest reports unavailable assets rather than pretending they exist. Export again after generation has finished if those assets are needed. A manual full export can be collected alongside incremental ZIPs; repeated identical events are deduplicated by the merge tool.

Merge the collection for analysis

```
.venv/bin/python tools/study_merge.py /path/to/collected-laptops --output /path/to/study-analysis
```

Run this from an installed checkout on the analysis computer. The input can be a directory containing nested laptop folders or individual ZIP paths. The command discovers complete ZIP bundles recursively and combines them by event ID. Conflicting content under the same event identity is treated as an error rather than silently choosing one version.

The analysis directory includes combined.sqlite3, event/session/media CSV files, exact JSONL and collected audio. media.csv links microphone clips to their participant sessions and local files. Use JSONL for exact nested payloads; CSV escapes formula-like text for spreadsheet use. Exceptionally large browser events may be stored as recording.event_chunk fragments. Reconstruct one logical event by joining its JSON strings in chunk_index order for the matching logical_event_id.

Check the merge result and each manifest’s missing-assets information before treating the collection as complete. Keep collected archives alongside the analysis output so a later script or interpretation can be rerun from the source records. Exporting or merging data does not automatically repopulate a dashboard’s Sessions library or generated-media store.

CHAPTER 21

Troubleshooting and verification

Start with the visible status message and the terminal running the server. Resolve installation and local browser issues before diagnosing physical equipment. The following checks do not require a robot or synthesizer unless the row explicitly refers to connected hardware.

Symptom	Action
The page will not open	Confirm the terminal still shows a running dashboard, then open http://127.0.0.1:8790 on that same laptop. If you chose --port, use that number. Check for an immediate startup error.
Address already in use on port 8790	Check whether the existing dashboard is the one you want. Stop it from its original terminal before launching another. ss -ltnp can identify a listener; inspect its command and PID before terminating anything.
An old var/dashboard.pid file exists	The normal installer does not manage a background service or PID file. A PID file from a previous manual launcher can be stale. Verify the process and command line before acting; do not blindly kill the number in the file.
Setup says Python has no version set	Use bash tools/setup_linux.sh, which selects /usr/bin/python3 for a new environment, or set PYTHON to a real Python 3.10+ executable. For an installed checkout, use .venv/bin/python directly.
The existing .venv is unusable	Setup deliberately keeps it intact. Stop the app, move that environment aside, and rerun setup with a suitable interpreter. Do not copy a .venv from another laptop.
The native build cannot find alsa/asoundlib.h	Install libasound2-dev and pkg-config or provide the matching extracted headers described in docs/linux.md. --skip-system does not install these dependencies.
No sound from Test sound	Check system volume, browser/site mute, the selected system output device and headphones. Click Test sound directly to activate browser audio. Test sound always uses browser audio; the separate Synth output setting determines where Synth mixer playback goes.
Playback waits for mapped input	Verify Signals from matches the source you configured. Choose Simulated demo signals for a standalone check, or select a recording/live input in Connections. An enabled mapping with missing input can pause sound.
New generation says the key is missing	Run .venv/bin/python tools/configure_api_key.py in a terminal, then restart the dashboard. Ensure it is the .env in this checkout and check whether an environment variable overrides it.
Provider authentication, access or quota error	Check the key’s expiry/revocation, endpoint permissions, account feature access and available credits in the provider dashboard. Provision a replacement with --replace when necessary. Repeated generation attempts can create additional paid requests.

Symptom	Action
Microphone recording is unavailable	Grant microphone permission for this local browser origin and select an input in the operating system. Installation and model readiness do not grant browser permission. Use typed text while resolving capture.
Transcription says the model is missing	Run .venv/bin/python tools/prepare_rap_transcription.py, or copy a complete var/whisper/small folder and rerun setup. The model and Python dependencies are both required.
Whisper will not load on this CPU	Read the installer’s load error. Confirm a complete model and a compatible installed transcription environment. Typed input is available while the local transcription issue is resolved.
hydractl list cannot open the ALSA sequencer	Check aconnect -l and access to /dev/snd/seq. Run under the ordinary logged-in desktop user with device access. A container may lack sound devices even when the host has them.
MIDI list contains only MIDI Through	No Hydrasynth is currently exposed to ALSA. That is expected without the synth connected. Check power, USB and device enumeration once the physical instrument is available.
Native MIDI works but browser MIDI does not	These are separate paths. Grant Web MIDI permission in a browser that supports it and choose the output in Connections. A native connection does not grant browser permission.
A native device disconnected	Reconnect the intended device, then stop and restart the native service. It deliberately does not select another instrument. Prefer a unique device name; ALSA numeric client:port addresses can change.
An ALSA audio route lists zero channels/rate	Named desktop/custom routes may have unknown capability metadata until metering starts. Use the actual UID to select the route; zero metadata alone is not proof of failure.
Direct hw: capture is busy	Another client may own the audio device. Close that capture client or select a shared/default PipeWire/PulseAudio ALSA route. The project does not require replacing the desktop audio server.
Robot serial permission denied	Check the selected serial device and user groups. On Ubuntu, dialout membership is commonly needed; log out/in after an administrator changes it. Do not run the dashboard as root.
Study is pending or reports a storage error	Keep the participant tab open. Check that the local server is running, the study folder is writable and disk space is available. Retry export only after pending data has reached local storage.
Saved settings seem missing after restart	Check browser profile and the exact hostname/port. Sessions are browser-local. Import an exported Sessions JSON when moving to another browser; make sure referenced audio assets also exist.

Verification without physical equipment

```
make native-test
make test
```

native-test runs both C self-tests and the Linux ALSA integration suite. It uses isolated virtual MIDI peers and synthetic audio rather than choosing a physical instrument or microphone. Tests that need unavailable ALSA sequencer access report a skip; a skipped test is not evidence that the host MIDI path works. make test adds the full Python/JavaScript dashboard suite and requires Node.js and permission to create local sockets.

For a prepared laptop, also verify Test sound, a bundled recording, a simulated signal and a saved voice demo. Confirm the researcher panel shows the expected installation ID and that a completed backup exists. These checks establish the local workflow without spending provider credits. A separate, intentionally initiated provider test is needed to establish generation access.

Physical MIDI reception, note-off/panic, real audio channels and levels, serial device identity, head calibration, emergency stop and robot stroke limits still require the actual rig. Use docs/hardware_acceptance.md and the hardware chapters of this manual when it is connected. Passing synthetic tests does not verify acoustic output or safe movement of an assembled robot.

CHAPTER 22

Reference glossary and verification

Use this glossary to connect the interface language with the different data, sound and hardware paths. The web application is also called Hydra Signal Lab; hydractl and hydra-audio are its separate native command-line tools.

Term	Meaning in this project
Recording input	Bundled or externally selected RAPP movement data. It is data used to drive mappings, not an audio recording of the synth.
Signal mix	A named normalized control signal built from one or more input fields. A mix must be assigned to a destination before it changes that sound parameter.
Manual value	The base setting of a parameter. Signal strength blends between this value and the mapped signal.
Stem	One separated component of a generated music track, such as bass or drums. Stem availability depends on successful preparation.
Browser audio	Sound generated or played by the browser through the computer output device. It needs a user gesture to start and is independent of native MIDI.
Web MIDI	Browser access to MIDI devices. Permission, saved-device selection and browser compatibility are separate from ALSA access in the native service.
ALSA	Linux audio and MIDI interfaces used by the native controller and input meter. Native MIDI uses the sequencer; the meter uses PCM input.
CC and NRPN	MIDI control encodings. Match the synth reception setting to the command type; a received controller number does not have the same meaning on every instrument.
Torque	The motor holding/driving force. Stopping a pattern, disconnecting a port and releasing torque are distinct actions.
Session version	A browser-local snapshot of application settings. Exported session JSON references audio and recordings; it is not a full study archive.
Study session	Recorded interaction events and linked media for one participant/session on a laptop. This is separate from the Sessions settings library.
Incremental backup	An archive of new events and new/changed media since previous backups. Retain the whole backup sequence.
Localhost	127.0.0.1 refers to the computer on which the browser is running. Six laptops each run their own server and store their own data.

How this edition was checked

The instructions were checked against the local README, Linux setup guide, robot control guide, hardware acceptance checklist, command-line help and current dashboard implementation on 20 October 2026. Screenshots were captured from the running application. Robot and synth hardware were absent; provider generation was not exercised.

The Linux native implementation previously passed both C self-tests and all 21 virtual MIDI/synthetic audio integration cases without skips. The dashboard suite passed 692 Python cases plus JavaScript behavior/syntax checks. Installer follow-up checks passed 14 credential-helper cases and 33 provider-service cases; the actual repeat-install path and offline model load passed. A first installation on a clean laptop, physical operation and provider account access remain separate acceptance checks.

Known native MIDI limitation: the current native panic/shutdown cleanup sends CC120, which the Hydrasynth mapping uses for ARP Octave. Review the native MIDI chapter before relying on cleanup behavior with a physical instrument. The browser All notes off path is different.

Maintaining this manual

The repository contains docs/manual-content.json, original screenshots in docs/manual-assets, the offline HTML/PDF outputs and tools/build_manual.py. Regenerate after changing source material or replacing screenshots. HTML generation requires only Python; PDF generation additionally requires ReportLab and DejaVu fonts. On Ubuntu/Debian, fonts-dejavu-core supplies those fonts.

```
# HTML only, using the application environment:
.venv/bin/python tools/build_manual.py --html-only

# Optional PDF authoring dependencies and complete rebuild:
.venv/bin/python -m pip install -r requirements-manual.txt
.venv/bin/python tools/build_manual.py
```

Supporting repository documents: README.md; docs/reference.md; docs/linux.md; docs/macos.md; docs/robot_control.md; docs/hardware_acceptance.md; docs/live_streaming.md; docs/movement_drums.md. Provider key permissions and quotas can change; check the current official documentation at https://elevenlabs.io/docs/overview/administration/workspaces/api-keys.